```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
using Logger = Sy.Core.Logger;
namespace Sy.P06.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S06_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot       Mock  →   →     T001④
    ///      Boot             UI T001b  Boot      =
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            BuildLoadingBadge();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        /// &lt;summary&gt;S00_Boot is a bare frame while the seven steps run and
        /// the main scene streams in. The koi+crane signature mark (X1706
        /// icon) rides the bottom-right corner for that window as the
        /// loading badge: a flourish that never blocks boot, skipped when
        /// art is absent (null = no dead UI), destroyed with the scene it
        /// lives in.&lt;/summary&gt;
        static void BuildLoadingBadge()
        {
            var art = ArtRuntime.Get("P06_icon_symbol_koi_crane");
            if (art == null) return;
            var canvasGo = new GameObject("LoadingBadgeCanvas", typeof(Canvas));
            var canvas = canvasGo.GetComponent&lt;Canvas&gt;();
            canvas.renderMode = RenderMode.ScreenSpaceOverlay;
            var badgeGo = new GameObject("LoadingBadge", typeof(RectTransform), typeof(Image));
            badgeGo.transform.SetParent(canvasGo.transform, false);
            var rt = (RectTransform)badgeGo.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(1f, 0f);
            rt.anchoredPosition = new Vector2(-90, 90);
```

```
            rt.sizeDelta = new Vector2(96, 96);
            var img = badgeGo.GetComponent&lt;Image&gt;();
            img.sprite = art;
            img.preserveAspect = true;
            img.raycastTarget = false;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
namespace Sy.P06.Boot
{
    /// &lt;summary&gt;
    /// Boot     T001
    /// ①Logger→②  →③Config→④Save→⑤ILocale→⑥Ads.Preload→⑦         [BOOT] n/7
    ///  ⑦        Play  =SceneManager.LoadSceneAsync Editor dry-run=EditorSceneManager.OpenScene
    /// X006      SY_MOCK
    /// &lt;/summary&gt;
    public sealed class BootSequence
    {
        public const int TotalSteps = 7;
        readonly Func&lt;bool&gt; _switchScene;
        readonly List&lt;string&gt; _lines = new();
        public BootSequence(Func&lt;bool&gt; switchScene)
        {
            _switchScene = switchScene;
        }
        public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
        public bool RunAll()
        {
            bool ok = true;
            ok &amp;= Run(1, "Logger", LoggerStep);
            ok &amp;= Run(2, "Time", TimeStep);
            ok &amp;= Run(3, "Config", ConfigStep);
            ok &amp;= Run(4, "Save", SaveStep);
            ok &amp;= Run(5, "ILocale", LocaleStep);
            ok &amp;= Run(6, "Ads.Preload", AdsStep);
```

```
            ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
            return ok;
        }
        bool Run(int n, string name, Func&lt;bool&gt; step)
        {
            bool result;
            try
            {
                result = step();
            }
            catch (Exception e)
            {
                Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
                result = false;
            }
            string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
            _lines.Add(line);
            Logger.Info("BOOT", line);
            return result;
        }
        static bool LoggerStep() =&gt; true; //              JS sink platform T001c
        static bool TimeStep()
        {
            var time = ServiceLocator.Get&lt;ITimeService&gt;();
            return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
        }
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock=    X009   manifest
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont); //         +
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
```

```
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.P06.Boot
{
    /// &lt;summary&gt;
    ///    Mock    SY_MOCK    X006     Mock X002a   AppID touristappid
    /// T001c  WX/TT     Channel   Editor      Boot
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static void RegisterAll()
        {
            if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; //
            ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
            ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
            ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
            ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
            ServiceLocator.Register&lt;IAdsService&gt;(new MockAdsService());
            ServiceLocator.Register&lt;IPlatformService&gt;(new MockPlatformService());   // T015-A share seam
            ServiceLocator.Register&lt;ILaunchInfo&gt;(new MockLaunchInfo());   // T015-B return seam (null query = orga
            ServiceLocator.Register&lt;IAlbum&gt;(new MockAlbum());   // T015-C album seam: mock records save paths; the
            Logger.Info("BOOT", "mock services registered (SY_MOCK)");
        }
    }
    /// T015-A mock platform seam (SY_MOCK): editor/dev sessions expose the
    /// share face so the scent-card button is real; the G4 platform bridge
    /// replaces this registration without touching any runtime call site.
    public sealed class MockPlatformService : IPlatformService
    {
        public Channel Channel =&gt; Channel.Editor;
        public System.Threading.Tasks.Task&lt;AuthResult&gt; Login()
            =&gt; System.Threading.Tasks.Task.FromResult(new AuthResult { Ok = true });
        public void Share(SharePayload payload)
        {
            Logger.Info("SHARE", "mock share captured: title=" + payload.Title + " image=" + payload.ImageUrl);
        }
        public UserInfo GetUserInfo() =&gt; new UserInfo();
    }
    public sealed class MockTimeService : ITimeService
    {
        public TimeSource Source =&gt; TimeSource.Mock;
        public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
        /// &lt;summary&gt;SY_MOCK X006                    TCB serverTime→HTTPS Date →   T001b   &lt;/summary&gt;
        public long ServerNowMs() =&gt; ClientNowMs();
```

```
        public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
    }
    public sealed class MockConfigService : IConfigService
    {
        public T GetTable&lt;T&gt;(string tableId) where T : class
            =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
        public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
        {
            table = null;
            return false;
        }
        public ConfigUpdateResult CheckUpdate(string manifestUrl)
            =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
        public void SwapPolicy(ConfigSwapPolicy policy) { }
        public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
    }
    public sealed class MockSaveService : ISaveService
    {
        const string Prefix = "sy_save_";
        public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
        public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
        /// &lt;summary&gt;X006             T001b v1→v2     &lt;/summary&gt;
        public SyncResult CloudSync() =&gt; new SyncResult(false, null);
    }
    public sealed class MockLocaleService : ILocaleService
    {
        public void SetLang(string lang) { }
        /// &lt;summary&gt; key  key      T001b cfg_i18n +  NUnit    &lt;/summary&gt;
        public string GetText(string key, params string[] args)
        {
            Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
            return key;
        }
        public FontSet GetFontSet()
            =&gt; new FontSet("LegacyRuntime.ttf", "LegacyRuntime.ttf", "LegacyRuntime.ttf");
    }
    public sealed class MockAdsService : IAdsService
    {
        public event Action&lt;AdLifecycle&gt; OnAdEvent;
        public Task&lt;RewardResult&gt; ShowRewarded(string pointId)
            =&gt; Task.FromResult(new RewardResult { Completed = true, Reason = "mock" });
        public void ShowInterstitial(string pointId) { }
        public void ShowBanner(string pointId) { }
        public void Preload(string pointId)
        {
            Logger.Info("ADS", "preload (mock): " + pointId);
            OnAdEvent?.Invoke(new AdLifecycle(AdLifecycleType.Fill, pointId, "mock"));
        }
    }
}
```

```
// MainUI.cs
using System;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;
using UnityEngine.UI;
using Sy.Core;
namespace Sy.P06.Newbie
{
    public static class NewbieSceneIds
    {
        public const string Main = "S06_Main";
        public const string Battle = "S06_Battle";
    }
    /// &lt;summary&gt;S06          →S06_Battle M5      ——
    /// W11b offline tip counter: settles the away-span into tips when returning
    /// to the main screen (parallel claim, clock never resets, cfg_idle rate law).
    /// The panel only appears when tips are pending - fresh installs and pre-boot
    /// states show nothing (no fake tips, no dead UI; the panel is pure idle
    /// economy, not an ad slot).&lt;/summary&gt;
    public sealed class MainUI : MonoBehaviour
    {
        TipCounter _tipCounter;
        ITimeService _clock;
        Sy.Telemetry.TelemetryService _shareTelemetry;   // T015-B: card_opened capture
        LetterPanel _letterPanel;   // T022: guest mailbox (weekly letters)
        Text _lettersBadge;         // T022: unseen-mail count on the mailbox button
        AboutPanel _aboutPanel;      // T024: compliance disclosure surface
        public void StartBusiness()
        {
            SceneManager.LoadScene(NewbieSceneIds.Battle);
        }
        void Awake()
        {
            Application.targetFrameRate = 30;
            MainArtInjector.Apply(transform, "P06_scene_01", "P06_unit_01");
            ResolveShareTelemetry();
            // T015-B K segment 2: read the launch query once per process; a
            // friend-card open fires evt_share(stage=card_opened) and arms a
            // one-shot pending recipe. Organic opens stay inactive (zero drift).
            ScentReturn.CaptureFromPlatform(_shareTelemetry);
            // U054b-B2: the same fixed launch query feeds the gift import
            // channel; organic opens stay Absent and never reach the toast.
            GiftImport.CaptureFromPlatform(Sy.Platform.AdCapPolicy.TodayKey());
            BuildTipPanel();
            BuildSharedReturnPanel();
            BuildSeasonShareButton();
            BuildLettersButton();
            BuildAboutButton();
            BuildGiftImportToast();
```

```
}
/// &lt;summary&gt;Boot-registered telemetry when the boot scene ran first
/// (BattleLauncher convention); the fallback keeps direct scene entry
/// working for tests and editor play.&lt;/summary&gt;
void ResolveShareTelemetry()
{
    if (!ServiceLocator.TryGet&lt;Sy.Telemetry.TelemetryService&gt;(out _shareTelemetry))
    {
        _shareTelemetry = new Sy.Telemetry.TelemetryService();
        _shareTelemetry.AddSink(new Sy.Telemetry.DebugLogSink());
    }
}
/// &lt;summary&gt;T015-B: the friend's-scent return banner. Only a real
/// src=share open with a resolvable recipe shows the face - anything
/// else (organic open, unknown recipe id) shows nothing, so headless
/// gates and fresh installs stay byte-identical. The button walks the
/// player straight into the shop to craft the same scent.&lt;/summary&gt;
void BuildSharedReturnPanel()
{
    if (!ScentReturn.OpenedFromShare || !ScentReturn.HasPendingRecipe) return;
    string nameKey = ScentCardComposer.NameKeyOf(ScentReturn.PendingRecipe);
    if (string.IsNullOrEmpty(nameKey)) return;   // unknown recipe: no dead banner
    string recipeName = StoryBoard.Text(nameKey);
    if (recipeName == nameKey) return;           // unresolved key: no key-name face
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
    var panel = MakePanel(canvas.transform, "SharedReturnPanel",
        new Vector2(0, 60), new Vector2(760, 340));
    MakeLabel(panel, StoryBoard.Text(ScentReturn.BannerTitleKey), 34,
        new Vector2(0, 122), new Vector2(720, 48), TipBrown);
    MakeLabel(panel, string.Format(StoryBoard.Text(ScentReturn.BannerLineKey), recipeName), 40,
        new Vector2(0, 28), new Vector2(720, 64), InkBrown);
    var go = MakeButton(panel, StoryBoard.Text(ScentReturn.BannerButtonKey),
        new Vector2(0, -102), new Vector2(360, 100));
    go.onClick.AddListener(() =&gt;
    {
        Destroy(panel.gameObject);
        StartBusiness();
    });
}
/// &lt;summary&gt;T015-E: the season panorama share face (three-piece #3).
/// The shop-identity brag button only appears when a platform service
/// can carry the payload (no dead buttons) and the label key resolves
/// to real copy; the compose-at-tap ReadPixels stall rides a
/// user-initiated menu frame, never a battle frame.&lt;/summary&gt;
void BuildSeasonShareButton()
{
    if (!ServiceLocator.TryGet&lt;Sy.Platform.IPlatformService&gt;(out _)) return;
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
```

```
    string label = StoryBoard.Text(SeasonCardComposer.ShareButtonKey);
    if (label == SeasonCardComposer.ShareButtonKey) return;   // unresolved key: no key-name face
    var go = MakeButton((RectTransform)canvas.transform, label,
        new Vector2(0, -780), new Vector2(420, 110));
    go.onClick.AddListener(OnSeasonShareTapped);
}
/// &lt;summary&gt;T022: the guest mailbox entry. Letters arrive with the
/// weekly flip, so the button shows once at least one letter is
/// unlocked (fresh installs already carry two) and the label key
/// resolves - no dead face, no key-name face. The badge counts
/// unseen unlocked mail only.&lt;/summary&gt;
void BuildLettersButton()
{
    if (LetterInbox.UnlockedLetters().Count == 0) return;
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
    string label = StoryBoard.Text(LetterPanel.KeyMailboxButton);
    if (label == LetterPanel.KeyMailboxButton) return;   // unresolved key: no key-name face
    var go = MakeButton((RectTransform)canvas.transform, label,
        new Vector2(-380, -780), new Vector2(260, 110));
    go.onClick.AddListener(OpenLetters);
    _lettersBadge = MakeLabel((RectTransform)go.transform, string.Empty, 34,
        new Vector2(78, 34), new Vector2(90, 52), MilkWhite);
    RefreshLettersBadge();
}
void OpenLetters()
{
    if (_letterPanel == null)
    {
        var canvas = FindObjectOfType&lt;Canvas&gt;();
        if (canvas == null) return;
        _letterPanel = gameObject.AddComponent&lt;LetterPanel&gt;();
        _letterPanel.Initialize(canvas);
        _letterPanel.Closed += RefreshLettersBadge;
    }
    _letterPanel.Open();
}
/// &lt;summary&gt;T024: the about/disclosure entry. A compliance face, so
/// unlike the mailbox it always shows when the canvas and the label
/// key resolve - no service dependency, no unlock curve. The panel
/// carries the AIGC disclosure, the rare-guest odds and the ad-cap
/// lines, all composed from the live law tables.&lt;/summary&gt;
void BuildAboutButton()
{
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
    string label = StoryBoard.Text(AboutPanel.KeyAboutButton);
    if (label == AboutPanel.KeyAboutButton) return;   // unresolved key: no key-name face
    var go = MakeButton((RectTransform)canvas.transform, label,
        new Vector2(380, -780), new Vector2(260, 110));
```

```
    go.onClick.AddListener(OpenAbout);
}
void OpenAbout()
{
    if (_aboutPanel == null)
    {
        var canvas = FindObjectOfType&lt;Canvas&gt;();
        if (canvas == null) return;
        _aboutPanel = gameObject.AddComponent&lt;AboutPanel&gt;();
        _aboutPanel.Initialize(canvas);
    }
    _aboutPanel.Open();
}
void RefreshLettersBadge()
{
    if (_lettersBadge == null) return;
    int unread = LetterInbox.UnreadCount();
    _lettersBadge.text = unread &gt; 0 ? unread.ToString() : string.Empty;
    _lettersBadge.gameObject.SetActive(unread &gt; 0);
}
/// &lt;summary&gt;T015-E/T015-F funnel: compose the panorama png at tap,
/// hand it to the platform service, and record pano_generated +
/// pano_shared for share attribution (offline data only, red line
/// 12). T015-F: the chat-preview face goes 1:1 - platform share
/// thumbnails center-crop 3:4 cards and lose the title band.&lt;/summary&gt;
void OnSeasonShareTapped()
{
    var spec = SeasonCardComposer.ComposeSquare();
    if (spec == null) return;
    if (!ServiceLocator.TryGet&lt;Sy.Platform.IPlatformService&gt;(out var platform)) return;
    var composer = new Sy.Social.ShareCardService();
    var result = composer.Compose(spec);
    if (!result.Ok)
    {
        Debug.LogWarning("[T015E] season card compose failed: " + result.FailReason);
        return;
    }
    _shareTelemetry.Track(Sy.Telemetry.EventIds.Share, new Dictionary&lt;string, string&gt;
    {
        { "stage", "pano_generated" },
    });
    // U054b-B1: the share link carries today's deterministic gift
    // package (gf= fragment) so a friend's open imports it and both
    // sides gain materials. Both grants land in the shared pantry
    // (PantryState) - the friend's via the B2 boot channel, the
    // sharer's once-daily reward right here. Honest absence: an
    // empty fragment leaves the query untouched.
    string dayKey = Sy.Platform.AdCapPolicy.TodayKey();
    var giftOracle = new GiftLinkOracle(PantryState.Stock);
    string giftQuery = giftOracle.ExportQuery(dayKey);
```

```
    string query = giftQuery.Length &gt; 0 ? spec.Query + "&amp;" + giftQuery : spec.Query;
    platform.Share(new Sy.Platform.SharePayload
    {
        Title = spec.Title,
        ImageUrl = result.Path,
        Query = query,
    });
    _shareTelemetry.Track(Sy.Telemetry.EventIds.Share, new Dictionary&lt;string, string&gt;
    {
        { "stage", "pano_shared" },
    });
    // U054b-B3: the sharer's once-daily reward banks into the
    // shared pantry the caravan reads; a cap-spent or full-pantry
    // day honestly shows nothing.
    if (giftOracle.TryGrantShareReward(dayKey))
        ShowShareRewardToast(dayKey, giftOracle);
    ShowGiftCodePanel(giftOracle.BuildCode(dayKey));
}
// U054b-B1 gift-code receipt face keys (cfg_i18n, dual copy).
const string GiftCodeTitleKey = "gift_code_title";
const string GiftCodeHintKey = "gift_code_hint";
const string GiftCodeCloseKey = "gift_code_close";
/// &lt;summary&gt;U054b-B1: the copyable gift-code face. After a carried
/// share fires, a compact panel shows today's deterministic code so
/// the sharer can also hand-copy it (no clipboard seam - the visible
/// text is the copy). Honest absence: no panel for an empty code or
/// an unresolved key, so headless gates stay byte-identical.&lt;/summary&gt;
void ShowGiftCodePanel(string code)
{
    if (string.IsNullOrEmpty(code)) return;
    string title = StoryBoard.Text(GiftCodeTitleKey);
    if (title == GiftCodeTitleKey) return;   // unresolved key: no key-name face
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
    var panel = MakePanel(canvas.transform, "GiftCodePanel",
        new Vector2(0, 160), new Vector2(760, 360));
    MakeLabel(panel, title, 34, new Vector2(0, 124), new Vector2(720, 48), TipBrown);
    MakeLabel(panel, code, 56, new Vector2(0, 46), new Vector2(720, 76), InkBrown);
    MakeLabel(panel, StoryBoard.Text(GiftCodeHintKey), 28,
        new Vector2(0, -34), new Vector2(700, 92), InkBrown);
    var close = MakeButton(panel, StoryBoard.Text(GiftCodeCloseKey),
        new Vector2(0, -118), new Vector2(340, 92));
    close.onClick.AddListener(() =&gt; Destroy(panel.gameObject));
}
// U054b-B3 sharer daily-reward toast key (cfg_i18n, dual copy).
const string GiftShareRewardKey = "gift_share_reward_toast";
/// &lt;summary&gt;U054b-B3: the sharer's daily-reward toast. Only a real
/// grant shows a face (cap spent / pantry full stay silent), the
/// line names the granted material, and an unresolved key or
/// material name means no face at all - headless gates stay
```

```
/// byte-identical.&lt;/summary&gt;
void ShowShareRewardToast(string dayKey, GiftLinkOracle oracle)
{
    string line = StoryBoard.Text(GiftShareRewardKey);
    if (line == GiftShareRewardKey) return;   // unresolved key: no key-name face
    string matName = MaterialNameOf(oracle.ComposePackage(dayKey)[0].MaterialId);
    if (matName.Length == 0) return;           // unresolved material name: no face
    var canvas = FindObjectOfType&lt;Canvas&gt;();
    if (canvas == null) return;
    var panel = MakePanel(canvas.transform, "ShareRewardToast",
        new Vector2(0, 430), new Vector2(640, 150));
    MakeLabel(panel, string.Format(line, matName), 30,
        new Vector2(0, 30), new Vector2(600, 76), InkBrown);
    var ok = MakeButton(panel, StoryBoard.Text(GiftCodeCloseKey),
        new Vector2(0, -42), new Vector2(280, 76));
    ok.onClick.AddListener(() =&gt; Destroy(panel.gameObject));
}
static string MaterialNameOf(int id)
{
    var materials = RecipeConfig.MaterialsBuiltIn();
    for (int i = 0; i &lt; materials.Length; i++)
    {
        if (materials[i].Id != id) continue;
        string name = StoryBoard.Text(materials[i].NameKey);
        return name == materials[i].NameKey ? "" : name;   // unresolved: empty
    }
    return "";
}
// U054b-B2 import outcome toast keys (cfg_i18n, dual copy).
const string GiftImportOkKey = "gift_import_ok_toast";
const string GiftImportDupKey = "gift_import_dup_toast";
const string GiftImportCapKey = "gift_import_cap_toast";
const string GiftImportFullKey = "gift_import_full_toast";
/// &lt;summary&gt;U054b-B2: the boot import toast. A gift-link open
/// names its outcome honestly - granted / duplicate / daily cap /
/// pantry full each get their own line, so the receiver always
/// knows what happened and why. Absent (organic open) and
/// Invalid (garbage link) show no face; an unresolved key shows
/// no key-name text - headless gates stay byte-identical.&lt;/summary&gt;
void BuildGiftImportToast()
{
    string key;
    switch (GiftImport.State)
    {
        case GiftLinkOracle.ImportState.Granted: key = GiftImportOkKey; break;
        case GiftLinkOracle.ImportState.Duplicate: key = GiftImportDupKey; break;
        case GiftLinkOracle.ImportState.DailyCap: key = GiftImportCapKey; break;
        case GiftLinkOracle.ImportState.Full: key = GiftImportFullKey; break;
        default: return;   // Absent / Invalid: honest silence
    }
```

```
        string line = StoryBoard.Text(key);
        if (line == key) return;   // unresolved key: no key-name face
        var canvas = FindObjectOfType&lt;Canvas&gt;();
        if (canvas == null) return;
        if (GiftImport.State == GiftLinkOracle.ImportState.Granted)
            line = string.Format(line, GiftImport.GrantedCount);
        var panel = MakePanel(canvas.transform, "GiftImportToast",
            new Vector2(0, 320), new Vector2(640, 170));
        MakeLabel(panel, line, 30, new Vector2(0, 30), new Vector2(600, 76), InkBrown);
        var ok = MakeButton(panel, StoryBoard.Text(GiftCodeCloseKey),
            new Vector2(0, -50), new Vector2(280, 80));
        ok.onClick.AddListener(() =&gt; Destroy(panel.gameObject));
    }
    /// &lt;summary&gt;W11b: build the offline tip panel only when tips are pending.
    /// Service lookups are guarded so opening S06_Main before boot (or in a
    /// bare harness) simply skips the panel - zero drift for headless gates.&lt;/summary&gt;
    void BuildTipPanel()
    {
        if (!ServiceLocator.TryGet&lt;ISaveService&gt;(out var save)) return;
        if (!ServiceLocator.TryGet&lt;ITimeService&gt;(out var clock)) return;
        _clock = clock;
        _tipCounter = new TipCounter(
            IdleConfig.Formula(IdleConfig.BuiltIn(), level: 0), save, clock.ServerNowMs());
        // T012-B shelf consumption point: owned shelves raise the
        // offline tip income by TipBonusPercent (launcher-side scale,
        // the TipCounter core the settle gate pins stays untouched).
        var facility = new FacilityOracle(FacilityConfig.BuiltIn(), FacilitySave.Load());
        double tipScale = 1.0 + facility.TipBonusPercent / 100.0;
        int coins = (int)Math.Floor(_tipCounter.PeekPending(clock.ServerNowMs()) * tipScale);
        if (coins &lt;= 0) return;
        var canvas = FindObjectOfType&lt;Canvas&gt;();
        if (canvas == null) return;
        var panel = MakePanel(canvas.transform, "TipPanel", new Vector2(0, -400), new Vector2(760, 300));
        MakeLabel(panel, NewbieConfig.TipPanelTitle, 34, new Vector2(0, 92), new Vector2(720, 48));
        var amount = MakeLabel(panel, string.Format(NewbieConfig.TipPanelAmount, coins), 58,
            new Vector2(0, 12), new Vector2(720, 84), InkBrown);
        var claim = MakeButton(panel, NewbieConfig.TipPanelClaim, new Vector2(0, -86), new Vector2(340, 96));
        claim.onClick.AddListener(() =&gt;
        {
            double got = _tipCounter.Claim(_clock.ServerNowMs()) * tipScale;
            amount.text = string.Format(NewbieConfig.TipPanelClaimed, (int)Math.Floor(got));
            claim.interactable = false;
        });
    }
    static readonly Color MistPink = new Color(0.831f, 0.647f, 0.647f, 0.55f);   // panel bg
    static readonly Color SageGreen = new Color(0.561f, 0.659f, 0.525f);        // claim button
    static readonly Color MilkWhite = new Color(0.961f, 0.941f, 0.910f);       // button label
    static readonly Color InkBrown = new Color(0.45f, 0.36f, 0.30f);            // amount text
    static readonly Color TipBrown = new Color(0.722f, 0.525f, 0.420f);         // title text
    static RectTransform MakePanel(Transform parent, string name, Vector2 pos, Vector2 size)
```

```
        {
            var go = new GameObject(name, typeof(RectTransform), typeof(Image));
            go.transform.SetParent(parent, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = pos;
            rt.sizeDelta = size;
            go.GetComponent&lt;Image&gt;().color = MistPink;
            return rt;
        }
        static Text MakeLabel(RectTransform parent, string text, int size, Vector2 pos, Vector2 dims, Color? color = nul
        {
            var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
            go.transform.SetParent(parent, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = pos;
            rt.sizeDelta = dims;
            var t = go.GetComponent&lt;Text&gt;();
            t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
            t.text = text;
            t.fontSize = size;
            t.alignment = TextAnchor.MiddleCenter;
            t.color = color ?? TipBrown;
            return t;
        }
        static Button MakeButton(RectTransform parent, string label, Vector2 pos, Vector2 size)
        {
            var go = new GameObject("ClaimButton", typeof(RectTransform), typeof(Image), typeof(Button));
            go.transform.SetParent(parent, false);
            var rt = (RectTransform)go.transform;
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = pos;
            rt.sizeDelta = size;
            go.GetComponent&lt;Image&gt;().color = SageGreen;
            var txt = MakeLabel(rt, label, 44, Vector2.zero, size);
            txt.color = MilkWhite;
            txt.raycastTarget = false;
            return go.GetComponent&lt;Button&gt;();
        }
    }
}
// AboutPanel.cs
using System;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P06.Newbie
{
    /// &lt;summary&gt;
    /// T024 compliance surface - the about/disclosure face on the main menu
```

```
/// (platform AIGC labeling rule + random-content disclosure). Every
/// disclosure line is composed FROM the live law tables (scent lure,
/// ad point) so the published numbers can never drift from the enforced
/// ones - the same construction guarantee the P02 odds panel uses.
/// Programmatic UGUI in the Morandi palette, zero scene-YAML drift,
/// presentation-only; the composer layer is pure static the headless
/// gate asserts without rendering.
/// &lt;/summary&gt;
public sealed class AboutPanel : MonoBehaviour
{
    public const string KeyAboutButton = "about_button";
    public const string KeyPanelTitle = "about_title";
    public const string KeyAigcLine = "about_aigc_line";
    public const string KeyOddsLine = "about_odds_line";
    public const string KeyAdCapLine = "about_adcap_line";
    public const string KeyAgeLine = "about_age_line";
    public const string KeyCloseLabel = "about_close";
    static readonly Color SageGreen = new Color(0.561f, 0.659f, 0.525f);
    static readonly Color MilkWhite = new Color(0.961f, 0.941f, 0.910f);
    static readonly Color OchreBrown = new Color(0.722f, 0.525f, 0.420f);
    static readonly Color InkBrown = new Color(0.45f, 0.36f, 0.30f);
    // ——— composer layer (pure static, gate-facing) ———
    /// &lt;summary&gt;AIGC explicit disclosure: art and audio assets in this
    /// game are AI-assisted (platform AI-generated-content labeling rule;
    /// the implicit platform-side declaration stays an admin item).&lt;/summary&gt;
    public static string AigcLine()
    {
        return StoryBoard.Text(KeyAigcLine);
    }
    /// &lt;summary&gt;Rare-guest odds disclosure. The published numbers are
    /// formatted straight out of the live scent-lure table, so the shown
    /// face and the enforced law are the same object - a hot-tuned table
    /// re-renders the disclosure with zero extra wiring.&lt;/summary&gt;
    public static string OddsLine()
    {
        var law = ScentLureConfig.BuiltIn();
        return string.Format(StoryBoard.Text(KeyOddsLine),
            Percent(law.RareBaseChance), Percent(law.RareWithAdChance), law.RarePityAt);
    }
    /// &lt;summary&gt;Ad-cap disclosure: the global daily ceiling formatted out
    /// of the live ad-point law (voluntary views only - no forced ad).&lt;/summary&gt;
    public static string AdCapLine()
    {
        var law = AdPointConfig.BuiltIn();
        return string.Format(StoryBoard.Text(KeyAdCapLine), law.GlobalCapDaily);
    }
    /// &lt;summary&gt;Age-rating disclosure (IAA casual default: 8+).&lt;/summary&gt;
    public static string AgeLine()
    {
        return StoryBoard.Text(KeyAgeLine);
```

```
}
static int Percent(double chance)
{
    return (int)Math.Round(chance * 100.0);
}
// ——— view layer ———
RectTransform _overlay;
public void Initialize(Canvas canvas)
{
    _overlay = new GameObject("AboutOverlay", typeof(RectTransform), typeof(Image)).GetComponent&lt;RectTransfor
    _overlay.transform.SetParent(canvas.transform, false);
    _overlay.anchorMin = Vector2.zero;
    _overlay.anchorMax = Vector2.one;
    _overlay.offsetMin = _overlay.offsetMax = Vector2.zero;
    var dim = _overlay.GetComponent&lt;Image&gt;();
    dim.sprite = MakeSquareSpriteShared();
    dim.color = new Color(0.35f, 0.30f, 0.27f, 0.92f);
    var paper = MakeCard(_overlay, "AboutPaper", Vector2.zero, new Vector2(940, 1180),
        new Color(MilkWhite.r, MilkWhite.g, MilkWhite.b, 0.97f));
    MakeText(paper, StoryBoard.Text(KeyPanelTitle), 64, new Vector2(0, 460),
        TextAnchor.MiddleCenter, 900, 110).color = OchreBrown;
    MakeText(paper, AigcLine(), 36, new Vector2(0, 250),
        TextAnchor.MiddleCenter, 820, 120).color = InkBrown;
    MakeText(paper, OddsLine(), 34, new Vector2(0, 60),
        TextAnchor.MiddleCenter, 820, 190).color = InkBrown;
    MakeText(paper, AdCapLine(), 34, new Vector2(0, -130),
        TextAnchor.MiddleCenter, 820, 110).color = InkBrown;
    MakeText(paper, AgeLine(), 34, new Vector2(0, -250),
        TextAnchor.MiddleCenter, 820, 110).color = InkBrown;
    MakeButton(paper, StoryBoard.Text(KeyCloseLabel), new Vector2(0, -440),
        new Vector2(420, 96), Close);
    _overlay.gameObject.SetActive(false);
}
public void Open()
{
    _overlay.gameObject.SetActive(true);
}
void Close()
{
    _overlay.gameObject.SetActive(false);
}
// ——— tools (programmatic UGUI, zero scene YAML) ———
static RectTransform MakeCard(RectTransform parent, string name, Vector2 pos, Vector2 size, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = size;
```

```
    var img = go.GetComponent&lt;Image&gt;();
    img.sprite = MakeSquareSpriteShared();
    img.color = color;
    return rt;
}
static Text MakeText(RectTransform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float
{
    var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(w, h);
    var t = go.GetComponent&lt;Text&gt;();
    t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
    t.text = text;
    t.fontSize = size;
    t.alignment = anchor;
    t.color = InkBrown;
    return t;
}
static void MakeButton(RectTransform parent, string label, Vector2 pos, Vector2 size, Action onClick)
{
    var go = new GameObject("ActionButton", typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = size;
    var img = go.GetComponent&lt;Image&gt;();
    img.sprite = MakeSquareSpriteShared();
    img.color = SageGreen;
    var txt = MakeText(rt, label, 40, Vector2.zero, TextAnchor.MiddleCenter, size.x, size.y);
    txt.color = MilkWhite;
    txt.raycastTarget = false;
    go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; onClick());
}
static Sprite _sharedSquare;
static Sprite MakeSquareSpriteShared()
{
    if (_sharedSquare == null)
    {
        var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
        var px = tex.GetPixels32();
        for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
        tex.SetPixels32(px);
        tex.Apply(false, false);
        _sharedSquare = Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
    }
    return _sharedSquare;
```

```
        }
    }
}
// AdPointTable.cs
using System;
using System.Collections.Generic;
namespace Sy.P06.Newbie
{
    /// &lt;summary&gt;Ad point kind: nine rewarded video points plus the single
    /// interstitial face (shop-rest settle only, the most restrained slot in
    /// the design table).&lt;/summary&gt;
    public enum AdPointType { Rewarded, Interstitial }
    /// &lt;summary&gt;One ad point row (pitch doc section six, V1.2 values).&lt;/summary&gt;
    public readonly struct AdPointEntry
    {
        public readonly string Id;
        public readonly string Category;
        public readonly string Surface;
        public readonly int CapPerDay;
        public readonly AdPointType Type;
        public readonly int Priority;
        public AdPointEntry(string id, string category, string surface,
            int capPerDay, AdPointType type, int priority)
        {
            Id = id;
            Category = category;
            Surface = surface;
            CapPerDay = capPerDay;
            Type = type;
            Priority = priority;
        }
    }
    /// &lt;summary&gt;Flat cfg_ad_point values (authoring table:
    /// Design/configs/P06/cfg_ad_point.json).&lt;/summary&gt;
    public readonly struct AdPointLaw
    {
        public readonly int Version;
        public readonly bool Disclosed;
        public readonly int GlobalCapDaily;
        public readonly int InterstitialCapDaily;
        public readonly double InterstitialCooldownSec;
        public readonly AdPointEntry[] Points;
        public AdPointLaw(int version, bool disclosed, int globalCapDaily,
            int interstitialCapDaily, double interstitialCooldownSec, AdPointEntry[] points)
        {
            Version = version;
            Disclosed = disclosed;
            GlobalCapDaily = globalCapDaily;
            InterstitialCapDaily = interstitialCapDaily;
            InterstitialCooldownSec = interstitialCooldownSec;
```

```
            Points = points;
        }
    }
    /// &lt;summary&gt;
    /// T014 ad point table (task book: 10 ad points = 9 rewarded + 1
    /// interstitial, pitch doc section six, V1.2). Mirrors the scent-lure/
    /// caravan/codex/facility table pattern: BuiltIn() is the embedded slice
    /// the live game runs on while Design/configs/P06/cfg_ad_point.json is the
    /// authoring table the ad-point gate validates against (drift guard: the
    /// two must agree row for row). Every rewarded row owns its category so
    /// the shared AdCapPolicy enforces the per-point cap on top of the 12/day
    /// global cap; the interstitial row carries its own 3/day cap and the
    /// 180s cooldown on the policy's interstitial face. Priority is a
    /// declared ordering for surfaces and analytics - the global cap stays
    /// first-come-first-served in v1.0, which the schema desc states openly.
    /// Id namespaces stay isolated from P02's funnel: the idle-earnings point
    /// is "idle_double", never P02's "offline_double", so a P06 call can
    /// never draw the P02 double budget (same isolation law the supply
    /// category already documents).
    /// &lt;/summary&gt;
    public static class AdPointConfig
    {
        static readonly AdPointEntry[] Embedded =
        {
            new AdPointEntry("idle_double", "idle_double", "harvest", 3, AdPointType.Rewarded, 50),
            new AdPointEntry("rare_guest", "rare_guest", "shop", 3, AdPointType.Rewarded, 60),
            new AdPointEntry("craft_speed", "craft_speed", "craft", 3, AdPointType.Rewarded, 40),
            new AdPointEntry("caravan_supply", "supply", "caravan", 3, AdPointType.Rewarded, 40),
            new AdPointEntry("recipe_hint", "recipe_hint", "craft", 1, AdPointType.Rewarded, 55),
            new AdPointEntry("sign_double", "sign_double", "signin", 1, AdPointType.Rewarded, 30),
            new AdPointEntry("expand_speed", "expand_speed", "expand", 1, AdPointType.Rewarded, 45),
            new AdPointEntry("story_preview", "story_preview", "story", 1, AdPointType.Rewarded, 20),
            new AdPointEntry("book_complete", "book_complete", "codex", 2, AdPointType.Rewarded, 25),
            new AdPointEntry("interstitial", "interstitial", "settle", 3, AdPointType.Interstitial, 10),
        };
        static readonly AdPointLaw EmbeddedLaw = new AdPointLaw(
            1, true, 12, 3, 180.0, Embedded);
        public static AdPointLaw BuiltIn() =&gt; EmbeddedLaw;
        /// &lt;summary&gt;Parse cfg_ad_point.json content (MiniJson: long/double).
        /// A malformed root returns false so callers fall back to the
        /// embedded table; missing fields parse as zero/empty so the
        /// zero-drift gate catches authoring gaps.&lt;/summary&gt;
        public static bool TryFromJson(string json, out AdPointLaw law)
        {
            law = default;
            var root = Sy.Core.MiniJson.Deserialize(json) as Dictionary&lt;string, object&gt;;
            if (root == null) return false;
            AdPointEntry[] points = null;
            if (root.TryGetValue("points", out var rawRows) &amp;&amp; rawRows is List&lt;object&gt; rows)
            {
```

```
                points = new AdPointEntry[rows.Count];
                for (int i = 0; i &lt; rows.Count; i++)
                {
                    if (!(rows[i] is Dictionary&lt;string, object&gt; row)) continue;
                    points[i] = new AdPointEntry(
                        AsString(row, "id"),
                        AsString(row, "category"),
                        AsString(row, "surface"),
                        AsInt(row, "capPerDay"),
                        ParseType(AsString(row, "type")),
                        AsInt(row, "priority"));
                }
            }
            law = new AdPointLaw(
                AsInt(root, "version"),
                AsBool(root, "disclosed"),
                AsInt(root, "globalCapDaily"),
                AsInt(root, "interstitialCapDaily"),
                AsDouble(root, "interstitialCooldownSec"),
                points);
            return true;
        }
        static AdPointType ParseType(string raw)
            =&gt; raw == "interstitial" ? AdPointType.Interstitial : AdPointType.Rewarded;
        static int AsInt(Dictionary&lt;string, object&gt; row, string key)
            =&gt; row.TryGetValue(key, out var v) &amp;&amp; v is long l ? (int)l : 0;
        static double AsDouble(Dictionary&lt;string, object&gt; row, string key)
        {
            if (!row.TryGetValue(key, out var v)) return 0;
            if (v is double d) return d;
            if (v is long l) return l;   // MiniJson yields long for integer literals
            return 0;
        }
        static bool AsBool(Dictionary&lt;string, object&gt; row, string key)
            =&gt; row.TryGetValue(key, out var v) &amp;&amp; v is bool b &amp;&amp; b;
        static string AsString(Dictionary&lt;string, object&gt; row, string key)
            =&gt; row.TryGetValue(key, out var v) &amp;&amp; v is string s ? s : "";
    }
}
// BattleLauncher.cs
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P06.Newbie
{
    /// &lt;summary&gt;
    /// S06_Battle       ShopSim+TutorialFlow+ShopView   Tick
    ///    Sy.Telemetry Editor =Debug sink T001e        +TCB
    /// HUD                     6
    /// &lt;/summary&gt;
    public sealed class BattleLauncher : MonoBehaviour
```

```
    {
        ShopSim _sim;
        TutorialFlow _tutorial;
        ShopView _view;
        StoryPanel _storyPanel;   // T009: guest-chapter story panel (weekly flip)
        CaravanOracle _caravan;      // T011: caravan supply loop (restock cycle)
        CaravanPanel _caravanPanel;  // T011: restock sheet (missing-material popup)
        CodexOracle _codex;          // T013: dual-codex progress (guests + recipes)
        CodexPanel _codexPanel;      // T013-B: the dual-codex page (tier reward claim)
        FacilityOracle _facility;    // T012: expansion engine (owned set from local save)
        FacilityPanel _facilityPanel; // T012-B: the store expansion page (purchase surface)
        RecipeHintOracle _hintOracle; // T014-B: recipe_hint point law (formal craft face)
        CraftSpeedOracle _speedOracle; // T014-B: craft_speed point law (formal brewing face)
        StoryPreviewOracle _previewOracle; // T014-B: story_preview point law (locked-chapter face)
        BookCompleteOracle _bookOracle; // T014-B: book_complete point law (codex claim doubler face)
        RareGuestBoostOracle _rareBoostOracle; // T014-B: rare_guest point law (VIP-lure boost face)
        SignDoubleOracle _signOracle;      // T014-B: sign_double point law (daily sign-in doubler face)
        InterstitialSettleOracle _interstitial; // T014-B: interstitial point law (shop-rest settle face)
        bool _formalSession;               // formal-loop flag: the tutorial stays ad-free (zero-ad red line before M5)
        SignInPanel _signPanel;            // T014-B: the daily sign-in sheet (D1-D7 streak surface)
        Sy.Idle.SignInSystem _signIn;       // T014-B: the claim law over the reward ladder
        bool _rareBoostArmed;           // T014-B: the next lure rolls at the boosted rare chance
        bool _lastLureReady;            // T014-B: latch for pushing lure-ready changes to the view
        Sy.Telemetry.TelemetryService _telemetry;
        Sy.Social.ShareCard _scentSpec;           // T015-A: 3:4 scent-card share face
        Sy.Social.ComposeResult _scentCard;
        Sy.Platform.IAlbum _album;              // T015-C: save-to-album seam (null = no bridge, save face stays hidden
        ScentAtmosphere _atmosphere;
        RareGuestOracle _rareOracle;      // T010 rare-guest law rider on the scent lure
        ReceptionOracle _reception;       // T002 reception-seat law (formal loop; tutorial stays inert)
        ScentLureValues _lureLaw;
        System.Random _rng;
        P06AudioHost _audioHost;          // X1675 matrix#44 slice-b: five-beat SFX host
        int _lastCoins = -1, _lastHearts = -1;
        void Awake()
        {
            Application.targetFrameRate = 30;
            var canvas = FindObjectOfType&lt;Canvas&gt;();
            _sim = new ShopSim();
            _view = gameObject.AddComponent&lt;ShopView&gt;();
            _view.Initialize(canvas, _sim);
#if DEVELOPMENT_BUILD || UNITY_EDITOR
            // U133 / doc 12 section 6: dev-only one-tap bug report. P06 has no
            // seed system - the context block carries the live shop state.
            Sy.Quality.BugReportService.Game = "P06";
            Sy.Quality.BugReportService.Version = Application.version;
            Sy.Quality.BugReportService.SeedInfo = "";
            Sy.Quality.BugReportService.ContextJson = () =&gt; P06QualityContext(_sim);
            Sy.Quality.BugReportButton.Attach(canvas.transform);
#endif
```

```
// T009: story panel rides the same canvas; the HUD story
// button is its only entry point (presentation-only).
_storyPanel = gameObject.AddComponent&lt;StoryPanel&gt;();
_storyPanel.Initialize(canvas);
// T011: caravan supply loop + restock sheet; the HUD caravan
// button opens it. The oracle owns the shared AdCapPolicy
// consume (P06's mock ad service has no cap logic - see the
// oracle doc for the T001c suite split). U054b-B3: the oracle
// reads the shared pantry (PantryState) so gift imports and
// share rewards land in the stock this sheet displays.
_caravan = new CaravanOracle(CaravanConfig.BuiltIn(), PantryState.Stock);
_caravanPanel = gameObject.AddComponent&lt;CaravanPanel&gt;();
_caravanPanel.Initialize(canvas, _caravan);
// T013: the dual codex fills as guests arrive and recipes are
// crafted (the tutorial order is the fox scholar + rain-green-hill,
// see NewbieConfig wiring ids). The panel is the claim surface; the
// launcher owns the wallet deposit - the panel never grants.
_codex = new CodexOracle(CodexConfig.BuiltIn());
_codexPanel = gameObject.AddComponent&lt;CodexPanel&gt;();
_codexPanel.Initialize(canvas, _codex);
// T012-B: the expansion engine boots from the local save (red
// line 12: PlayerPrefs authority) and the panel is its purchase
// surface - the launcher owns the wallet spend and the save write.
_facility = new FacilityOracle(FacilityConfig.BuiltIn(), FacilitySave.Load());
_facilityPanel = gameObject.AddComponent&lt;FacilityPanel&gt;();
_facilityPanel.Initialize(canvas, _facility);
// T014-B slice 1: the recipe_hint ad point (V1.2: 1/day, craft
// surface). The oracle owns the AdCapPolicy consume (mock ad
// service has no cap logic); the tutorial keeps the hint face
// hidden (zero-ad red line before M5) - the formal craft batch
// lights it via SetHintFace(true).
_hintOracle = new RecipeHintOracle(AdPointConfig.BuiltIn());
_view.HintTapped += OnHintTapped;
_view.SetHintFace(false);
// T014-B slice 2: the craft_speed ad point (V1.2: 3/day, brewing
// wait surface). The oracle owns the AdCapPolicy consume (mock ad
// service has no cap logic); the grant skips the brewing wait so
// the recipe card flips right away. The tutorial keeps the speed
// face hidden (zero-ad red line before M5) - the formal craft
// batch lights it via SetSpeedFace(true).
_speedOracle = new CraftSpeedOracle(AdPointConfig.BuiltIn());
_view.SpeedTapped += OnSpeedTapped;
_view.SetSpeedFace(false);
// T014-B slice 3: the story_preview ad point (V1.2: 1/day,
// locked-chapter face). The oracle owns the AdCapPolicy consume
// (mock ad service has no cap logic); the grant opens the locked
// chapter's opening as a read-only preview (clues stay sealed).
// The tutorial keeps the preview face hidden (zero-ad red line
// before M5) - the formal story batch lights it via
// SetPreviewFace(true).
```

```
_previewOracle = new StoryPreviewOracle(AdPointConfig.BuiltIn());
_storyPanel.PreviewTapped += OnPreviewTapped;
_storyPanel.SetPreviewFace(false);
// T015-D: the chapter-detail keepsake share face. Sharing is not
// an ad point, so the face lights whenever a platform service can
// carry the payload (no dead buttons) - the tutorial may share too.
_storyPanel.StoryShareTapped += OnStoryShareTapped;
_storyPanel.SetStoryShareFace(Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IPlatformService&gt;(out _));
// T014-B slice 4: the book_complete ad point (V1.2: 2/day, the
// codex tier-claim doubler face). The oracle owns the AdCapPolicy
// consume (mock ad service has no cap logic); the grant doubles
// the claim it rides on. The tutorial keeps the doubler face
// hidden (zero-ad red line before M5) - the formal codex batch
// lights it via SetDoubleFace(true).
_bookOracle = new BookCompleteOracle(AdPointConfig.BuiltIn());
_codexPanel.DoubleClaimRequested += OnCodexDoubleRequested;
_codexPanel.SetDoubleFace(false);
// T014-B slice 5: the rare_guest ad point (V1.2: 3/day,
// VIP-lure-ready street face). The oracle owns the AdCapPolicy
// consume (mock ad service has no cap logic); the grant arms the
// next scent lure to roll the rare arrival at the boosted 0.8
// chance instead of the 0.05 base. The tutorial keeps the boost
// face hidden (zero-ad red line before M5) - the formal street
// batch lights it via SetRareFace(true).
_rareBoostOracle = new RareGuestBoostOracle(AdPointConfig.BuiltIn());
_view.RareTapped += OnRareBoostTapped;
_view.SetRareFace(false);
// T014-B slice 6: the sign_double ad point (V1.2: 1/day, the
// daily sign-in doubler face). The sign-in sheet opens from the
// battle HUD - the main screen stays ad-free by design - and the
// oracle owns the AdCapPolicy consume (mock ad service has no
// cap logic); the grant doubles the sign-in reward deposit. The
// tutorial keeps the doubler face hidden (zero-ad red line
// before M5) - the formal sign-in batch lights it via
// SetDoubleFace(true).
_signOracle = new SignDoubleOracle(AdPointConfig.BuiltIn());
_signIn = new Sy.Idle.SignInSystem(NewbieConfig.SignInRewardTable);
_signPanel = gameObject.AddComponent&lt;SignInPanel&gt;();
_signPanel.Initialize(canvas);
_signPanel.SetDoubleFace(false);
// T014-B slice 7: the interstitial point (V1.2: 3/day + 180s
// cooldown, shop-rest settle face). No button, no reward - the
// platform suite owns the cooldown + cap consume, so the oracle
// stays a thin formal-session gate.
_interstitial = new InterstitialSettleOracle(AdPointConfig.BuiltIn());
// T014-B formal-loop slice: the settle "reopen" reloads the
// battle scene, so the durable session count (PlayerPrefs,
// red line 12) is the only state that survives the reload.
// Session 0 is the tutorial - every ad face stays dark on that
// boot (zero-ad red line before M5); every later boot is a
```

```
// formal session and lights all six faces at once (each one
// still self-gates on its own live moment via SyncView).
_formalSession = FormalSessionSave.Load() &gt;= 1;
if (_formalSession)
{
    _view.SetHintFace(true);
    _view.SetSpeedFace(true);
    _storyPanel.SetPreviewFace(true);
    _codexPanel.SetDoubleFace(true);
    _view.SetRareFace(true);
    _signPanel.SetDoubleFace(true);
}
// T006 scent atmosphere: orders fill the street, the VIP follows the scent.
_atmosphere = new ScentAtmosphere();
_view.SetScentSource(_atmosphere);
_atmosphere.VipLureTriggered += OnVipLureTriggered;
// T002 reception slice: the formal loop seats every lured VIP
// (rare guests reuse their stage seconds as the stay) and pays
// a departure tip scaled by the shelf family; a full house
// turns the guest away. The tutorial keeps the oracle inert so
// the pinned settle math never moves - capacity reads the
// facility engine live, so a mid-run expansion is effective
// on the very next lure.
_reception = new ReceptionOracle(
    _formalSession,
    () =&gt; _facility.GuestCapacity,
    () =&gt; _facility.TipBonusPercent);
_view.SetSeatedFace(0, _reception.Capacity);
// T010 rare-guest law (embedded slice; VipRareCheck guards zero
// drift against Design/configs/P06/cfg_scent_lure.json).
_lureLaw = ScentLureConfig.BuiltIn();
_rareOracle = new RareGuestOracle(_lureLaw);
_rng = new System.Random();
// X1675 matrix#44 slice-b: scene-local audio host with the five
// teaching-beat SFX wiring (accept click / invest feedback triad
// / craft success / codex reveal / settle tips). The launcher
// owns this mapping (P02 X813 paradigm) so the editor census
// mirrors the exact production path through the probe ledger.
// Null-safe law: a missing clip stays silent in any build state.
_audioHost = P06AudioHost.Ensure("bgm_shop", 0.6f);
_sim.OrderAccepted += () =&gt; _audioHost.PlaySfx("sfx_ui_click", 0.8f);
_sim.FeedbackGiven += (slot, materialId, state, trueSlot)
    =&gt; _audioHost.PlaySfx(FeedbackSfxKey(state), 0.85f);
_sim.RecipeCardShown += () =&gt; _audioHost.PlaySfx("sfx_craft_success", 0.9f);
_sim.CodexUnlocked += () =&gt; _audioHost.PlaySfx("sfx_codex", 0.9f);
_sim.Settled += () =&gt; _audioHost.PlaySfx("sfx_tips", 0.9f);
_telemetry = new Sy.Telemetry.TelemetryService();
_telemetry.AddSink(new TelemetryDebugSink());
_tutorial = new TutorialFlow(
    _sim,
```

```
                hint =&gt;
                {
                    Debug.Log("[HINT] " + hint);
                    _view.SetHint(hint);
                },
                step =&gt; _telemetry.Track(Sy.Telemetry.EventIds.NewbieStep,
                    new Dictionary&lt;string, string&gt; { { "step", step.ToString() } }));
            _view.AcceptRequested += OnAcceptRequested;
            _view.MaterialTapped += OnMaterialTapped;
            _view.SocialTapped += OnSocialTapped;
            _view.StoryTapped += OnStoryTapped;
            _view.CaravanTapped += OnCaravanTapped;
            _view.CodexTapped += OnCodexTapped;
            _view.FacilityTapped += OnFacilityTapped;
            _view.SignTapped += OnSignTapped;
            _caravanPanel.SupplyRequested += OnCaravanSupplyRequested;
            _codexPanel.ClaimRequested += OnCodexClaimRequested;
            _facilityPanel.PurchaseRequested += OnFacilityPurchaseRequested;
            _signPanel.SignInRequested += OnSignInClaimRequested;
            _signPanel.DoubleSignInRequested += OnSignInDoubleRequested;
            _sim.StateChanged += OnStateChanged;
            _sim.FeedbackGiven += OnFeedbackGiven;
            _sim.OrderCompleted += OnOrderCompleted;
            _sim.StarGained += OnStarGained;
            _sim.RecipeCardShown += OnRecipeCardShown;
        _view.ShareRequested += OnCardShareRequested;   // T015-A: card-face share funnel
        _view.SaveRequested += OnCardSaveRequested;     // T015-C: card-face save-to-album funnel
        _view.AlbumRetryTapped += OnAlbumRetryTapped;  // T015-C: denied-save retry funnel
        }
#if DEVELOPMENT_BUILD || UNITY_EDITOR
        /// &lt;summary&gt;U133 / doc 12 section 6: dev-only report context, fresh at tap
        /// time (BR-*.json context block = live shop state for reproduction).&lt;/summary&gt;
        static string P06QualityContext(ShopSim sim)
        {
            if (sim == null) return "{}";
            return "{\"coins\":" + sim.Coins + ",\"hearts\":" + sim.Hearts
                + ",\"stars\":" + sim.Stars + ",\"activeSlot\":" + sim.ActiveSlot
                + ",\"stateTimer\":" + sim.StateTimer.ToString("F1", System.Globalization.CultureInfo.InvariantCulture)
                + ",\"correctInvests\":" + sim.CorrectInvests + "}";
        }
#endif
        void Update()
        {
            _sim.Tick(Time.deltaTime);
            _atmosphere.Tick(Time.deltaTime);
            _view.SyncView(_sim);
            // T002 reception: advance every seated guest - departures pay
            // their tip into the wallet and pop on the HUD (formal loop
            // only; the tutorial oracle never seats or departs anyone).
            var departures = _reception.Tick(Time.deltaTime);
```

```
    if (departures.Count &gt; 0)
    {
        int tipTotal = 0;
        for (int i = 0; i &lt; departures.Count; i++) tipTotal += departures[i].TipCoins;
        _sim.Deposit(tipTotal, 0);
        _view.PlayIncomePop(tipTotal, 0);
        _view.SetSeatedFace(_reception.SeatedCount, _reception.Capacity);
        // T019 portrait pair: the seat clears when the house empties
        // (the counter HUD keeps carrying the live seat truth).
        if (_reception.SeatedCount == 0) _view.HideSeatedGuest();
    }
    if (_sim.Coins != _lastCoins) { _lastCoins = _sim.Coins; _view.SetCoins(_lastCoins); }
    if (_sim.Hearts != _lastHearts) { _lastHearts = _sim.Hearts; _view.SetHearts(_lastHearts); }
    // T014-B: push lure-ready changes to the rare face so the boost
    // offer only surfaces on the live street moment.
    bool lureReady = _atmosphere.VipLureReady;
    if (lureReady != _lastLureReady) { _lastLureReady = lureReady; _view.SetRareFaceLive(lureReady); }
}
void OnAcceptRequested()
{
    _sim.AcceptOrder();
}
void OnMaterialTapped(int materialId)
{
    _sim.TryInvest(materialId);
}
void OnSocialTapped()
{
    _view.SetHint("      ——                ");
}
void OnStoryTapped()
{
    // X1680 audio face: UI tap group - panel-open click beat.
    _audioHost.PlaySfx("sfx_ui_click", 0.8f);
    _storyPanel.Open();
}
void OnCodexTapped()
{
    _audioHost.PlaySfx("sfx_ui_click", 0.8f);
    _codexPanel.Open();
}
void OnFacilityTapped()
{
    _audioHost.PlaySfx("sfx_ui_click", 0.8f);
    _facilityPanel.Open(_sim.Coins, _sim.Hearts);
}
void OnFacilityPurchaseRequested(int facilityId)
{
    // The oracle owns the affordability law; the wallet spends only
    // after a successful purchase, so coins never drain without a
```

```
    // row joining the owned set (and an owned row never re-charges).
    if (!_facility.TryPurchase(facilityId, _sim.Coins, _sim.Hearts, out _, out _))
    {
        _facilityPanel.Refresh(_sim.Coins, _sim.Hearts);
        return;
    }
    var entry = _facility.Entry(facilityId);
    _sim.TrySpend(entry.CostCoins, entry.CostHearts);
    FacilitySave.Save(_facility.OwnedIds());
    _facilityPanel.Refresh(_sim.Coins, _sim.Hearts);
}
void OnCodexClaimRequested(CodexSide side)
{
    // The ledger grants once per tier step; the deposit lands in the
    // shop wallet and the HUD picks the diff up on its next sync.
    if (_codex.TryClaimTierReward(side, out int coins, out int hearts))
    {
        _sim.Deposit(coins, hearts);
        _codexPanel.Refresh();
    }
}
void OnCaravanTapped()
{
    // X1680 audio face: UI tap group - panel-open click beat.
    _audioHost.PlaySfx("sfx_ui_click", 0.8f);
    _caravanPanel.Open();
    if (_caravan.MissingCount(_caravan.Law.ShowcaseRecipeId) &gt; 0)
        TrackCaravan("offer");
}
async void OnCaravanSupplyRequested()
{
    string day = Sy.Platform.AdCapPolicy.TodayKey();
    if (!_caravan.CanOffer(day)) return;
    if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
    {
        var result = await ads.ShowRewarded(_caravan.Law.PointId);
        if (!result.Completed)
        {
            TrackCaravan("ad_fail");
            return;
        }
    }
    // The oracle consumes the supply-category slot itself (P06 mock
    // suite has no cap logic; T001c moves the consume into the suite).
    if (_caravan.TryCompleteSupply(day))
    {
        _caravanPanel.Refresh();
        TrackCaravan("accept");
    }
    else
```

```
    {
        TrackCaravan("blocked");
    }
}
void TrackCaravan(string state)
{
    _telemetry.Track(Sy.Telemetry.EventIds.Ad, new Dictionary&lt;string, string&gt;
    {
        { "state", state },
        { "pointId", _caravan.Law.PointId },
        { "reason", "caravan_supply" },
        { "missing", _caravan.MissingCount(_caravan.Law.ShowcaseRecipeId).ToString() },
    });
}
// ──── T014-B recipe_hint consumption (formal craft face) ────
async void OnHintTapped()
{
    string day = Sy.Platform.AdCapPolicy.TodayKey();
    bool crafting = _sim.State == ShopSim.ShopState.Crafting;
    if (!_hintOracle.CanOffer(day, crafting, _sim.ActiveSlot))
    {
        TrackHint("blocked");
        return;
    }
    TrackHint("offer");
    if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
    {
        var result = await ads.ShowRewarded(_hintOracle.PointId);
        if (!result.Completed)
        {
            TrackHint("ad_fail");
            return;
        }
    }
    // The oracle consumes the recipe_hint slot itself (mock ad
    // service has no cap logic - CaravanOracle integration debt).
    if (_hintOracle.TryGrant(day, crafting, _sim.ActiveSlot, out int slot))
    {
        _view.ShowHint(slot);
        TrackHint("accept");
    }
    else
    {
        TrackHint("blocked");
    }
}
void TrackHint(string state)
{
    _telemetry.Track(Sy.Telemetry.EventIds.Ad, new Dictionary&lt;string, string&gt;
    {
```

```
        { "state", state },
        { "pointId", _hintOracle.PointId },
        { "reason", "recipe_hint" },
    });
}
// ——— T014-B craft_speed consumption (formal brewing face) ———
async void OnSpeedTapped()
{
    string day = Sy.Platform.AdCapPolicy.TodayKey();
    bool brewing = _sim.State == ShopSim.ShopState.Brewing;
    if (!_speedOracle.CanOffer(day, brewing))
    {
        TrackSpeed("blocked");
        return;
    }
    TrackSpeed("offer");
    if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
    {
        var result = await ads.ShowRewarded(_speedOracle.PointId);
        if (!result.Completed)
        {
            TrackSpeed("ad_fail");
            return;
        }
    }
    // The oracle consumes the craft_speed slot itself (mock ad
    // service has no cap logic - CaravanOracle integration debt);
    // the grant lands as a brewing-wait skip, so the card flips now.
    if (_speedOracle.TryGrant(day, brewing) &amp;&amp; _sim.TrySkipBrew())
    {
        TrackSpeed("accept");
    }
    else
    {
        TrackSpeed("blocked");
    }
}
void TrackSpeed(string state)
{
    _telemetry.Track(Sy.Telemetry.EventIds.Ad, new Dictionary&lt;string, string&gt;
    {
        { "state", state },
        { "pointId", _speedOracle.PointId },
        { "reason", "craft_speed" },
    });
}
// ——— T014-B story_preview consumption (locked-chapter face) ———
async void OnPreviewTapped(ChapterEntry chapter)
{
    string day = Sy.Platform.AdCapPolicy.TodayKey();
```

```
    // Face law: only a still-locked chapter can preview - live rows
    // open free, so a flipped week never burns ad budget.
    bool locked = !StoryBoard.IsLive(chapter);
    if (!_previewOracle.CanOffer(day, locked))
    {
        TrackPreview("blocked");
        return;
    }
    TrackPreview("offer");
    if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
    {
        var result = await ads.ShowRewarded(_previewOracle.PointId);
        if (!result.Completed)
        {
            TrackPreview("ad_fail");
            return;
        }
    }
    // The oracle consumes the story_preview slot itself (mock ad
    // service has no cap logic - CaravanOracle integration debt);
    // the grant renders the read-only preview, clues stay sealed.
    if (_previewOracle.TryGrant(day, locked))
    {
        _storyPanel.ShowPreview(chapter);
        TrackPreview("accept");
    }
    else
    {
        TrackPreview("blocked");
    }
}
void TrackPreview(string state)
{
    _telemetry.Track(Sy.Telemetry.EventIds.Ad, new Dictionary&lt;string, string&gt;
    {
        { "state", state },
        { "pointId", _previewOracle.PointId },
        { "reason", "story_preview" },
    });
}
// ——— T014-B book_complete consumption (codex claim doubler face) ———
async void OnCodexDoubleRequested(CodexSide side)
{
    string day = Sy.Platform.AdCapPolicy.TodayKey();
    // Face law: only a still-pending tier claim gates the offer -
    // a claimed-out book never burns ad budget.
    if (!_bookOracle.CanOffer(day, _codex.HasPendingTierReward(side)))
    {
        TrackBookDouble("blocked");
        return;
```

```
    }
    TrackBookDouble("offer");
    if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
    {
        var result = await ads.ShowRewarded(_bookOracle.PointId);
        if (!result.Completed)
        {
            TrackBookDouble("ad_fail");
            return;
        }
    }
    // Re-check the face after the await: a plain claim during the ad
    // flips the pending state off - the oracle then denies without
    // consuming, so budget never burns without a claim to double.
    bool stillPending = _codex.HasPendingTierReward(side);
    if (_bookOracle.TryGrant(day, stillPending)
        &amp;&amp; _codex.TryClaimTierReward(side, out int coins, out int hearts))
    {
        _sim.Deposit(coins * 2, hearts * 2);
        _codexPanel.Refresh();
        TrackBookDouble("accept");
    }
    else
    {
        TrackBookDouble("blocked");
    }
}
void TrackBookDouble(string state)
{
    _telemetry.Track(Sy.Telemetry.EventIds.Ad, new Dictionary&lt;string, string&gt;
    {
        { "state", state },
        { "pointId", _bookOracle.PointId },
        { "reason", "book_complete" },
    });
}
async void OnRareBoostTapped()
{
    string day = Sy.Platform.AdCapPolicy.TodayKey();
    // Face law: only a lure-ready street gates the offer - a cold
    // street never burns ad budget.
    if (!_rareBoostOracle.CanOffer(day, _atmosphere.VipLureReady))
    {
        TrackRareBoost("blocked");
        return;
    }
    TrackRareBoost("offer");
    if (Sy.Core.ServiceLocator.TryGet&lt;Sy.Platform.IAdsService&gt;(out var ads))
    {
        var result = await ads.ShowRewarded(_rareBoostOracle.PointId);
```

```
    streetImgs[i].color = new Color(SageGreen.r, SageGreen.g, SageGreen.b, 0f);
    phase[i] = i * 0.7f;
}
float mistClock = 0f;
while (true)
{
    mistClock += Time.deltaTime;
    float conc = _atmosphere != null ? Mathf.Clamp01(_atmosphere.Concentration) : 0f;
    if (_lureBurst &gt; 0f) _lureBurst -= Time.deltaTime;
    bool surge = _lureBurst &gt; 0f;
    float doorAlpha = Mathf.Min(0.55f + conc * 0.25f + (surge ? 0.2f : 0f), 1f);
    // 8 fps 3-frame loop, staggered per puff for a rolling mist
    if (hasMist)
    {
        int mf = (int)(mistClock * 8f) % 3;
        for (int i = 0; i &lt; puffs.Length; i++)
        {
            var f = mistFrames[(mf + i) % 3];
            if (doorImgs[i].sprite != f) doorImgs[i].sprite = f;
        }
        for (int i = 0; i &lt; street.Length; i++)
        {
            var f = mistFrames[(mf + i) % 3];
            if (streetImgs[i].sprite != f) streetImgs[i].sprite = f;
        }
    }
    // door puffs: ambient drift up, denser while the scent lingers
    for (int i = 0; i &lt; puffs.Length; i++)
    {
        var pos = puffs[i].anchoredPosition;
        pos.y += (55f + conc * 26f) * Time.deltaTime;
        pos.x += (i % 2 == 0 ? 14f : -10f) * Time.deltaTime;
        puffs[i].anchoredPosition = pos;
        if (pos.y &gt; 350f) ResetPuff(puffs[i], 0f);
        var dc = doorImgs[i].color;
        dc.a = doorAlpha;
        doorImgs[i].color = dc;
    }
    // street puffs: flow out of the door toward the street, scaled by concentration
    for (int i = 0; i &lt; street.Length; i++)
    {
        var sc = streetImgs[i].color;
        var streetBase = _rareSurge ? WarmAmber : (hasMist ? Color.white : SageGreen);   // T010: amber stre
        sc.r = streetBase.r; sc.g = streetBase.g; sc.b = streetBase.b;
        sc.a = conc * (surge ? 0.85f : 0.45f);
        streetImgs[i].color = sc;
        phase[i] += Time.deltaTime * (0.6f + conc * 1.2f);
        var pos = street[i].anchoredPosition;
        pos.x -= (26f + conc * 58f) * Time.deltaTime;
        pos.y = -100f + Mathf.Sin(phase[i]) * 26f;
```

```
            if (pos.x &lt; -520f) pos.x = -350f;
            street[i].anchoredPosition = pos;
            float s = 26f + Mathf.Sin(phase[i] * 0.8f) * 6f + (surge ? 10f : 0f);
            street[i].sizeDelta = new Vector2(s, s);
        }
        yield return null;
    }
}
static void ResetPuff(RectTransform puff, float delayX)
{
    puff.anchoredPosition = new Vector2(-350 + (delayX * 40f) % 160f, -40f);
}
// ——— HUD    ———
public void SetCoins(int c) =&gt; _coinText.text = "   " + c;
public void SetHearts(int h) =&gt; _heartText.text = "   " + h;
public void SetHint(string hint) =&gt; _hintText.text = hint;
public void HideOpeningOverlay() =&gt; _openingOverlay.gameObject.SetActive(false);
public void ShowCraftingPanel(bool show) =&gt; _craftingPanel.gameObject.SetActive(show);
public void ShowBubble(bool show) =&gt; _bubble.gameObject.SetActive(show);
public void ShowVipBanner(bool show) =&gt; _vipBanner.gameObject.SetActive(show);
public void ShowSettle(int coins, int hearts, int stars, int codex)
{
    _settlePanel.gameObject.SetActive(true);
    _settleStats.text = $"   {coins} ·    {hearts} ·     {stars}  ·    {codex}  \n       {NewbieConfig.RecipeNam
}
// ———    ———
RectTransform MakeCanvasChild(string name, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    var img = rt.GetComponent&lt;Image&gt;();
    img.sprite = _square;
    img.color = color;
    return rt;
}
RectTransform MakeChildOf(RectTransform parent, string name, Vector2 pos, Vector2 size)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = size;
    return rt;
}
RectTransform MakeOverlayPanel(string name, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
```

```
    go.transform.SetParent(_canvas.transform, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = Vector2.zero;
    rt.anchorMax = Vector2.one;
    rt.offsetMin = rt.offsetMax = Vector2.zero;
    var image = go.GetComponent&lt;Image&gt;();
    image.sprite = _square;
    image.color = color;
    return rt;
}
/// &lt;summary&gt;               +    &lt;/summary&gt;
RectTransform MakeOverlayBubble()
{
    var rt = MakeCanvasChild("Bubble", MilkWhite);
    rt.anchoredPosition = new Vector2(0, 180);
    rt.sizeDelta = new Vector2(460, 220);
    return rt;
}
Button MakePanelButton(RectTransform parent, string label, Vector2 pos, Action onClick)
{
    var go = new GameObject(label, typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(300, 96);
    var image = go.GetComponent&lt;Image&gt;();
    image.sprite = _square;
    image.color = SageGreen;
    MakeText(go.transform, label, 42, Vector2.zero, TextAnchor.MiddleCenter, 300, 96).color = MilkWhite;
    var button = go.GetComponent&lt;Button&gt;();
    button.onClick.AddListener(() =&gt; onClick());
    return button;
}
static Text MakeText(Transform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float h)
{
    var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(w, h);
    var t = go.GetComponent&lt;Text&gt;();
    t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
    t.text = text;
    t.fontSize = size;
    t.alignment = anchor;
    t.color = new Color(0.91f, 0.89f, 0.85f);
    t.supportRichText = true;
    return t;
```

```
        }
        static Sprite MakeSquareSprite()
        {
            var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
            var px = tex.GetPixels32();
            for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
            tex.SetPixels32(px);
            tex.Apply(false, false);
            return Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
        }
    }
}
// SignDoubleOracle.cs
using System;
namespace Sy.P06.Newbie
{
    /// &lt;summary&gt;
    /// T014-B slice 6: the sign_double consumption law (ad point table
    /// row "sign_double", V1.2 cap 1/day, face = today's daily sign-in
    /// still unclaimed). The grant doubles that day's sign-in reward
    /// deposit - the IAA upsell rides the D1-D7 retention moment. The
    /// oracle owns the shared AdCapPolicy consume exactly like the book
    /// doubler (P06's mock ad service has no cap logic); it stays pure C#
    /// with injected day keys so the whole law is headless-verifiable.
    /// The tutorial keeps the doubler face hidden (zero-ad red line
    /// before M5); the formal sign-in batch lights it via
    /// SetDoubleFace(true). Placement ruling: the sign-in sheet lives on
    /// the battle HUD because the main screen stays ad-free by design
    /// (MainUI owns no ad slots).
    /// &lt;/summary&gt;
    public sealed class SignDoubleOracle
    {
        readonly AdPointEntry _point;
        readonly Sy.Platform.AdCapPolicy _adCap;
        public SignDoubleOracle(AdPointLaw law)
            : this(law, Sy.Platform.AdCapPolicy.DefaultDailyCap) { }
        /// &lt;summary&gt;Gates construct oracles with a small global cap to pin
        /// the global/category interplay without touching the policy type
        /// from the editor assembly (book-doubler pattern).&lt;/summary&gt;
        public SignDoubleOracle(AdPointLaw law, int globalDailyCap)
        {
            _point = default;
            var pts = law.Points;
            for (int i = 0; i &lt; pts.Length; i++)
                if (pts[i].Id == "sign_double") { _point = pts[i]; break; }
            _adCap = new Sy.Platform.AdCapPolicy(globalDailyCap);
        }
        public string PointId =&gt; _point.Id;
        public string Category =&gt; _point.Category;
        public int DailyCap =&gt; _point.CapPerDay;
```

```
        public int UsedToday(string dayKey)
            =&gt; _adCap.UsedCategoryToday(dayKey, _point.Category);
        /// &lt;summary&gt;Pre-offer gate: only an unclaimed daily sign-in with
        /// category budget left - no doubler on a signed day, no
        /// dead-end taps.&lt;/summary&gt;
        public bool CanOffer(string dayKey, bool signInPendingToday)
            =&gt; ValidFace(signInPendingToday)
            &amp;&amp; _adCap.HasCategoryBudget(dayKey, _point.Category);
        /// &lt;summary&gt;Watched-ad grant: consumes one sign_double slot
        /// (1/day) and reports success - the caller then signs in at
        /// double value. Denies without consuming whenever the face is
        /// invalid (already signed) or the day's budget is gone.&lt;/summary&gt;
        public bool TryGrant(string dayKey, bool signInPendingToday)
        {
            if (_point.Id == null) return false;
            if (!ValidFace(signInPendingToday)) return false;
            return _adCap.TryConsumeRewarded(dayKey, _point.Category);
        }
        static bool ValidFace(bool signInPendingToday) =&gt; signInPendingToday;
    }
}
// SignInPanel.cs
using System;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P06.Newbie
{
    /// &lt;summary&gt;Live face state of the daily sign-in: whether today is
    /// still unclaimed, the reward a sign-in would pay, the saved streak
    /// and the streak a sign-in would produce. Pure snapshot - the gate
    /// pins the builder law, the panel only renders it.&lt;/summary&gt;
    public readonly struct SignInFaceModel
    {
        public readonly bool PendingToday;
        public readonly long TodayReward;
        public readonly int CurrentStreak;
        public readonly int NextStreak;
        public SignInFaceModel(bool pendingToday, long todayReward, int currentStreak, int nextStreak)
        {
            PendingToday = pendingToday;
            TodayReward = todayReward;
            CurrentStreak = currentStreak;
            NextStreak = nextStreak;
        }
    }
    /// &lt;summary&gt;
    /// T014-B slice 6: the daily sign-in sheet (the sign_double ad
    /// point's consumption surface). Placement ruling: the sheet opens
    /// from the battle HUD because the main screen stays ad-free by
    /// design (MainUI owns no ad slots) - the sign-in streak is the
```

```
/// D1-D7 retention loop and the doubler rides the claim moment.
/// Same split as the codex page: the panel is presentation-only,
/// BuildModel is the pure static law the gate asserts, every display
/// string resolves through the shipped strings table (no Chinese in
/// source) and the panel never grants by itself - the launcher owns
/// the wallet deposit, the save write and the ad funnel.
/// &lt;/summary&gt;
public sealed class SignInPanel : MonoBehaviour
{
    public const string KeyTitle = "signin_panel_title";
    public const string KeyStreakLabel = "signin_streak_label";
    public const string KeyRewardLabel = "signin_reward_label";
    public const string KeyClaimButton = "signin_claim_button";
    public const string KeyClaimedLabel = "signin_claimed_label";
    public const string KeyCloseLabel = "signin_close_label";
    public const string KeyDoubleButton = "sign_double_button";
    public event Action SignInRequested;
    public event Action DoubleSignInRequested;
    RectTransform _overlay;
    RectTransform _sheetRoot;
    bool _doubleFace;
    static readonly Color SageGreen = new Color(0.561f, 0.659f, 0.525f);    // grey-green #8FA886
    static readonly Color MilkWhite = new Color(0.961f, 0.941f, 0.910f);   // milk white #F5F0E8
    static readonly Color OchreBrown = new Color(0.722f, 0.525f, 0.420f);  // ochre #B8866B
    static readonly Color ColdGrey = new Color(0.620f, 0.660f, 0.720f);    // cold Morandi grey-blue
    static readonly Color InkBrown = new Color(0.45f, 0.36f, 0.30f);        // ink-brown copy
    /// &lt;summary&gt;Pure face law: the sign-in result of (lastDay, today,
    /// streak) over the reward table - the same SignInSystem law the
    /// launcher claims through, so the sheet can never disagree with
    /// the wallet. today == lastDay (&gt;= 0) means already signed.&lt;/summary&gt;
    public static SignInFaceModel BuildModel(long lastSignDayIndex, long todayIndex, int streak, long[] rewardTable)
    {
        var r = new Sy.Idle.SignInSystem(rewardTable).SignIn(lastSignDayIndex, todayIndex, streak);
        return new SignInFaceModel(r.Accepted, r.Reward, streak, r.Streak);
    }
    public void Initialize(Canvas canvas)
    {
        _overlay = new GameObject("SignInOverlay", typeof(RectTransform), typeof(Image)).GetComponent&lt;RectTransfo
        _overlay.transform.SetParent(canvas.transform, false);
        _overlay.anchorMin = Vector2.zero;
        _overlay.anchorMax = Vector2.one;
        _overlay.offsetMin = _overlay.offsetMax = Vector2.zero;
        var dim = _overlay.GetComponent&lt;Image&gt;();
        dim.sprite = MakeSquareSpriteShared();
        dim.color = new Color(0.35f, 0.30f, 0.27f, 0.92f);
        var paper = MakeCard(_overlay, "SignInPaper", Vector2.zero, new Vector2(940, 900),
            new Color(MilkWhite.r, MilkWhite.g, MilkWhite.b, 0.97f));
        _sheetRoot = MakeContainer(paper, "SheetRoot");
        _overlay.gameObject.SetActive(false);
    }
```

```
/// &lt;summary&gt;Opens the sheet with a fresh face snapshot pushed by
/// the launcher (the launcher owns the save + clock reads).&lt;/summary&gt;
public void Open(SignInFaceModel model)
{
    if (_overlay == null) return;
    _overlay.gameObject.SetActive(true);
    Refresh(model);
}
/// &lt;summary&gt;Arms or hides the sign_double doubler face (T014-B:
/// the launcher locks it false through the tutorial - zero-ad red
/// line before M5 - and the formal sign-in batch lights it).&lt;/summary&gt;
public void SetDoubleFace(bool armed)
{
    _doubleFace = armed;
}
/// &lt;summary&gt;Re-renders the sheet from a face snapshot so the
/// claim state, streak and doubler never go stale.&lt;/summary&gt;
public void Refresh(SignInFaceModel model)
{
    if (_overlay == null) return;
    Clear(_sheetRoot);
    MakeText(_sheetRoot, StoryBoard.Text(KeyTitle), 72, new Vector2(0, 300),
        TextAnchor.MiddleCenter, 940, 110).color = OchreBrown;
    MakeText(_sheetRoot, string.Format(StoryBoard.Text(KeyStreakLabel), model.CurrentStreak),
        40, new Vector2(0, 190), TextAnchor.MiddleCenter, 900, 70).color = InkBrown;
    // reward line: an unclaimed day shows what signing in pays,
    // a signed day shows the come-back-tomorrow copy
    MakeText(_sheetRoot, model.PendingToday
            ? string.Format(StoryBoard.Text(KeyRewardLabel), model.TodayReward)
            : StoryBoard.Text(KeyClaimedLabel),
        46, new Vector2(0, 90), TextAnchor.MiddleCenter, 900, 84).color = InkBrown;
    var claim = MakeButton(_sheetRoot,
        model.PendingToday ? StoryBoard.Text(KeyClaimButton) : StoryBoard.Text(KeyClaimedLabel),
        new Vector2(0, -40), new Vector2(640, 110), model.PendingToday);
    if (model.PendingToday)
        claim.onClick.AddListener(() =&gt; SignInRequested?.Invoke());
    // T014-B sign_double doubler: visible only when the formal
    // face is armed and today is actually unclaimed - the
    // tutorial never shows an ad button (zero-ad red line before
    // M5) and a signed day shows no dead upsell.
    bool showDouble = _doubleFace &amp;&amp; model.PendingToday;
    if (showDouble)
    {
        MakeButton(_sheetRoot, StoryBoard.Text(KeyDoubleButton),
            new Vector2(0, -160), new Vector2(640, 72), true)
            .onClick.AddListener(() =&gt; DoubleSignInRequested?.Invoke());
    }
    MakeButton(_sheetRoot, StoryBoard.Text(KeyCloseLabel),
        new Vector2(0, showDouble ? -268 : -200), new Vector2(420, 92), true)
        .onClick.AddListener(() =&gt; _overlay.gameObject.SetActive(false));
```

```
}
// ——— helpers (programmatic UGUI, zero scene-YAML) ———
static void Clear(RectTransform root)
{
    for (int i = root.childCount - 1; i &gt;= 0; i--)
        Destroy(root.GetChild(i).gameObject);
}
static RectTransform MakeContainer(RectTransform parent, string name)
{
    var go = new GameObject(name, typeof(RectTransform));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = Vector2.zero;
    rt.offsetMin = rt.offsetMax = Vector2.zero;
    return rt;
}
static RectTransform MakeCard(RectTransform parent, string name, Vector2 pos, Vector2 size, Color color)
{
    var go = new GameObject(name, typeof(RectTransform), typeof(Image));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = size;
    var img = go.GetComponent&lt;Image&gt;();
    img.sprite = MakeSquareSpriteShared();
    img.color = color;
    return rt;
}
static Text MakeText(RectTransform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float
{
    var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
    rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(w, h);
    var t = go.GetComponent&lt;Text&gt;();
    t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
    t.text = text;
    t.fontSize = size;
    t.alignment = anchor;
    t.color = InkBrown;
    return t;
}
static Button MakeButton(RectTransform parent, string label, Vector2 pos, Vector2 size, bool enabled)
{
    var go = new GameObject("ActionButton", typeof(RectTransform), typeof(Image), typeof(Button));
    go.transform.SetParent(parent, false);
    var rt = (RectTransform)go.transform;
```

```
            rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
            rt.anchoredPosition = pos;
            rt.sizeDelta = size;
            var img = go.GetComponent&lt;Image&gt;();
            img.sprite = MakeSquareSpriteShared();
            img.color = enabled ? SageGreen : new Color(ColdGrey.r, ColdGrey.g, ColdGrey.b, 0.55f);
            var txt = MakeText(rt, label, 38, Vector2.zero, TextAnchor.MiddleCenter, size.x, size.y);
            txt.color = MilkWhite;
            txt.raycastTarget = false;
            return go.GetComponent&lt;Button&gt;();
        }
        static Sprite _sharedSquare;
        static Sprite MakeSquareSpriteShared()
        {
            if (_sharedSquare == null)
            {
                var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
                var px = tex.GetPixels32();
                for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
                tex.SetPixels32(px);
                tex.Apply(false, false);
                _sharedSquare = Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
            }
            return _sharedSquare;
        }
    }
    /// &lt;summary&gt;Daily sign-in persistence (red line 12: local
    /// PlayerPrefs authority, zero server). Format: "lastDay,streak",
    /// empty = never signed.&lt;/summary&gt;
    public static class SignInSave
    {
        public const string StateKey = "p06_signin_state";
        public static void Load(out long lastSignDayIndex, out int streak)
        {
            lastSignDayIndex = -1;
            streak = 0;
            var raw = UnityEngine.PlayerPrefs.GetString(StateKey, "");
            if (string.IsNullOrEmpty(raw)) return;
            var parts = raw.Split(',');
            if (parts.Length &gt;= 1 &amp;&amp; long.TryParse(parts[0], out var d)) lastSignDayIndex = d;
            if (parts.Length &gt;= 2 &amp;&amp; int.TryParse(parts[1], out var s)) streak = s;
        }
        public static void Save(long lastSignDayIndex, int streak)
        {
            UnityEngine.PlayerPrefs.SetString(StateKey, lastSignDayIndex + "," + streak);
            UnityEngine.PlayerPrefs.Save();
        }
        /// &lt;summary&gt;Test probe: wipes the persisted sign-in state.&lt;/summary&gt;
        public static void ClearForTests()
        {
```

```
            UnityEngine.PlayerPrefs.DeleteKey(StateKey);
        }
    }
}
// StoryBoard.cs
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.P06.Newbie
{
    /// &lt;summary&gt;One row of the story list: a chapter plus its live/locked
    /// state under the current week (weekly flip driver).&lt;/summary&gt;
    public readonly struct ChapterRow
    {
        public readonly ChapterEntry Chapter;
        public readonly bool IsLive;
        public ChapterRow(ChapterEntry chapter, bool isLive)
        {
            Chapter = chapter;
            IsLive = isLive;
        }
    }
    /// &lt;summary&gt;
    /// T009 story service + weekly flip driver. Joins GuestConfig chapters
    /// with RecipeConfig clue ids and the shipped strings table, and
    /// resolves which chapters are live for the current week. The week
    /// source is an injectable provider (live builds can hook the calendar
    /// week; gates pin exact weeks) so the whole release curve stays
    /// headless-verifiable. Display text comes from the cfg_i18n TextAsset
    /// shipped under Resources (authoring table: Design/configs/P06/
    /// cfg_i18n.json; GuestCheck U1 pins the two byte-for-byte) so runtime
    /// code never hardcodes display copy.
    /// &lt;/summary&gt;
    public static class StoryBoard
    {
        static Func&lt;int&gt; _weekProvider = DefaultWeek;
        static Dictionary&lt;string, string&gt; _strings;
        static bool _stringsLoaded;
        static int DefaultWeek() =&gt; 0;
        /// &lt;summary&gt;Pins the week source (gates/tests). Passing null
        /// restores the default provider.&lt;/summary&gt;
        public static void SetWeekProvider(Func&lt;int&gt; provider)
        {
            _weekProvider = provider ?? DefaultWeek;
        }
        public static int CurrentWeek =&gt; _weekProvider();
        /// &lt;summary&gt;Weekly flip driver: a chapter is live when playable at
        /// the current week (stock chapters flip in place at their
        /// releaseWeek and never regress).&lt;/summary&gt;
        public static bool IsLive(ChapterEntry chapter)
```

```
    {
        return GuestConfig.IsPlayableAt(chapter, CurrentWeek);
    }
    public static int LiveChapterCount()
    {
        return GuestConfig.CountPlayableAt(GuestConfig.ChaptersBuiltIn(), CurrentWeek);
    }
    /// &lt;summary&gt;Story list for one guest: every chapter they host with
    /// its live/locked state (locked rows keep releaseWeek so the UI
    /// can show the "next week" label).&lt;/summary&gt;
    public static List&lt;ChapterRow&gt; RowsForGuest(int guestId)
    {
        var rows = new List&lt;ChapterRow&gt;();
        var chapters = GuestConfig.ChaptersBuiltIn();
        for (int i = 0; i &lt; chapters.Length; i++)
            if (chapters[i].GuestId == guestId)
                rows.Add(new ChapterRow(chapters[i], IsLive(chapters[i])));
        return rows;
    }
    /// &lt;summary&gt;Resolved clue lines for a chapter, in clue order:
    /// finished display sentences from the strings table, ready for the
    /// story UI (no raw keys, no manual joins).&lt;/summary&gt;
    public static List&lt;string&gt; ClueLines(ChapterEntry chapter)
    {
        var lines = new List&lt;string&gt;();
        for (int i = 0; i &lt; chapter.ClueIds.Length; i++)
            lines.Add(Text("clue_" + chapter.ClueIds[i] + "_text"));
        return lines;
    }
    /// &lt;summary&gt;Display text lookup; falls back to the raw key so a
    /// missing string stays visible and debuggable instead of blank.&lt;/summary&gt;
    public static string Text(string key)
    {
        return TryText(key, out var value) ? value : key;
    }
    public static bool TryText(string key, out string value)
    {
        if (!_stringsLoaded) LoadStrings();
        if (_strings != null &amp;&amp; _strings.TryGetValue(key, out value)) return true;
        value = null;
        return false;
    }
    static void LoadStrings()
    {
        _stringsLoaded = true;
        var asset = Resources.Load&lt;TextAsset&gt;("cfg_i18n");
        if (asset == null) return;
        GuestConfig.TryI18nKeysFromJson(asset.text, out _strings);
    }
}
```

```
}
// StoryCardComposer.cs
using Sy.Social;
namespace Sy.P06.Newbie
{
    /// &lt;summary&gt;
    /// T015-D story-completion card: the chapter the player just finished
    /// becomes a 3:4 keepsake png (K engine reuse, xiaohongshu-ready share
    /// of the three-piece set). The face carries the chapter title, the
    /// host guest and the closing line - real story data only (the season
    /// panorama slice waits for its art + season mechanics; no fabricated
    /// seasons on a shipped face). Copy resolves through the i18n table at
    /// compose time; the query carries the chapter id for share attribution.
    /// &lt;/summary&gt;
    public static class StoryCardComposer
    {
        public const string ShareQueryPrefix = "src=share&amp;story=";
        public const string TaglineKey = "story_card_line";
        public static ShareCard FromChapter(ChapterEntry chapter)
        {
            string title = StoryBoard.Text(chapter.TitleKey);
            string guest = StoryBoard.Text(GuestNameKeyOf(chapter.GuestId));
            string close = StoryBoard.Text(chapter.CloseKey);
            return AutoCardComposer.Card(
                ScentCardComposer.RecipeTemplateKey,   // shared wood-paper brand face
                title,
                new[] { guest, close, StoryBoard.Text(TaglineKey) },
                StoryBoard.Text(ScentCardComposer.FooterKey),
                ShareQueryPrefix + chapter.Id);
        }
        /// &lt;summary&gt;Id resolution with the no-dead-end law: an unknown id
        /// falls back to the first live chapter so the share face never
        /// renders an empty promise.&lt;/summary&gt;
        public static ShareCard FromChapterId(int chapterId)
        {
            var chapters = GuestConfig.ChaptersBuiltIn();
            for (int i = 0; i &lt; chapters.Length; i++)
                if (chapters[i].Id == chapterId) return FromChapter(chapters[i]);
            return chapters.Length &gt; 0 ? FromChapter(chapters[0]) : null;
        }
        static string GuestNameKeyOf(int guestId)
        {
            var guests = GuestConfig.GuestsBuiltIn();
            for (int i = 0; i &lt; guests.Length; i++)
                if (guests[i].Id == guestId) return guests[i].NameKey;
            return string.Empty;
        }
    }
}
// StoryPanel.cs
```

```
using System;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
namespace Sy.P06.Newbie
{
    /// &lt;summary&gt;One story-list section: a guest host plus their chapter
    /// rows under the current week (live or locked).&lt;/summary&gt;
    public readonly struct GuestSection
    {
        public readonly GuestEntry Guest;
        public readonly List&lt;ChapterRow&gt; Rows;
        public GuestSection(GuestEntry guest, List&lt;ChapterRow&gt; rows)
        {
            Guest = guest;
            Rows = rows;
        }
    }
    /// &lt;summary&gt;
    /// T009 story panel - the in-game wiring for the StoryBoard service
    /// (RowsForGuest / ClueLines / Text). Programmatic UGUI in the Morandi
    /// palette, zero scene-YAML drift, presentation-only (never touches the
    /// shop state machine). Every display string resolves through the
    /// shipped strings table so no Chinese lives in source; the list layer
    /// is a pure static model (BuildSections) the headless gate asserts
    /// without rendering. Locked rows keep their releaseWeek so the UI can
    /// promise when the chapter flips live.
    /// &lt;/summary&gt;
    public sealed class StoryPanel : MonoBehaviour
    {
        public const string KeyTitle = "story_panel_title";
        public const string KeyWeeklyHint = "story_weekly_hint";
        public const string KeyLockedLabel = "story_locked_label";
        public const string KeyCloseLabel = "story_close_label";
        public const string KeyPreviewButton = "story_preview_button";
        public const string KeyPreviewNote = "story_preview_note";
        public const string KeyStoryShareButton = "story_share_button";
        /// &lt;summary&gt;T014-B3: a locked row's preview button fired. The launcher
        /// owns the ad funnel (offer/ad_fail/accept/blocked); the panel only
        /// renders - it never grants.&lt;/summary&gt;
        public event Action&lt;ChapterEntry&gt; PreviewTapped;
        /// &lt;summary&gt;T015-D: the chapter-detail share button fired. The
        /// launcher owns the compose + platform share funnel; the panel only
        /// renders the face it was armed with.&lt;/summary&gt;
        public event Action&lt;ChapterEntry&gt; StoryShareTapped;
        bool _previewArmed;   // T014-B3: locked rows show the preview button only when armed
        bool _storyShareArmed;   // T015-D: live chapter details show the share button only when armed
        Sprite _square;
        RectTransform _overlay;
        RectTransform _listRoot;
```

```
RectTransform _detailRoot;
static readonly Color SageGreen = new Color(0.561f, 0.659f, 0.525f);    //    #8FA886
static readonly Color MilkWhite = new Color(0.961f, 0.941f, 0.910f);   //    #F5F0E8
static readonly Color OchreBrown = new Color(0.722f, 0.525f, 0.420f);  //    #B8866B
static readonly Color ColdGrey = new Color(0.620f, 0.660f, 0.720f);    //  ·
static readonly Color InkBrown = new Color(0.45f, 0.36f, 0.30f);        //
/// &lt;summary&gt;Pure list model for the gate and the view: one section
/// per guest that hosts chapters, each with their rows under the
/// current week. Guests without chapters stay out of the list
/// (no dead UI, no fake promises).&lt;/summary&gt;
public static List&lt;GuestSection&gt; BuildSections()
{
    var sections = new List&lt;GuestSection&gt;();
    var guests = GuestConfig.GuestsBuiltIn();
    for (int i = 0; i &lt; guests.Length; i++)
    {
        var rows = StoryBoard.RowsForGuest(guests[i].Id);
        if (rows.Count &gt; 0)
            sections.Add(new GuestSection(guests[i], rows));
    }
    return sections;
}
public void Initialize(Canvas canvas)
{
    _square = MakeSquareSpriteShared();
    _overlay = new GameObject("StoryOverlay", typeof(RectTransform), typeof(Image)).GetComponent&lt;RectTransfor
    _overlay.transform.SetParent(canvas.transform, false);
    _overlay.anchorMin = Vector2.zero;
    _overlay.anchorMax = Vector2.one;
    _overlay.offsetMin = _overlay.offsetMax = Vector2.zero;
    var dim = _overlay.GetComponent&lt;Image&gt;();
    dim.sprite = _square;
    dim.color = new Color(0.35f, 0.30f, 0.27f, 0.92f);
    var paper = MakeCard(_overlay, "StoryPaper", Vector2.zero, new Vector2(1020, 1700),
        new Color(MilkWhite.r, MilkWhite.g, MilkWhite.b, 0.97f));
    _listRoot = MakeContainer(paper, "ListRoot");
    _detailRoot = MakeContainer(paper, "DetailRoot");
    _detailRoot.gameObject.SetActive(false);
    _overlay.gameObject.SetActive(false);
}
/// &lt;summary&gt;Opens the panel on the chapter list, rebuilt from the
/// current week so live/locked states are always fresh.&lt;/summary&gt;
public void Open()
{
    _overlay.gameObject.SetActive(true);
    ShowList();
}
/// &lt;summary&gt;T014-B3: arms/disarms the locked-row preview face. The
/// launcher keeps it locked in the tutorial (zero-ad red line before
/// M5); the formal story batch lights it.&lt;/summary&gt;
```

```
public void SetPreviewFace(bool armed)
{
    _previewArmed = armed;
}
/// &lt;summary&gt;T015-D: arms/disarms the chapter-detail share face. The
/// launcher arms it only when a platform service can carry the
/// payload (no dead buttons); sharing is not an ad point, so the
/// tutorial face may light too.&lt;/summary&gt;
public void SetStoryShareFace(bool armed)
{
    _storyShareArmed = armed;
}
void ShowList()
{
    _detailRoot.gameObject.SetActive(false);
    Clear(_listRoot);
    _listRoot.gameObject.SetActive(true);
    MakeText(_listRoot, StoryBoard.Text(KeyTitle), 76, new Vector2(0, 690),
        TextAnchor.MiddleCenter, 1020, 110).color = OchreBrown;
    MakeText(_listRoot, StoryBoard.Text(KeyWeeklyHint), 32, new Vector2(0, 588),
        TextAnchor.MiddleCenter, 1020, 60).color = new Color(0.62f, 0.55f, 0.50f);
    float y = 440f;
    var sections = BuildSections();
    for (int s = 0; s &lt; sections.Count; s++)
    {
        var section = sections[s];
        MakeText(_listRoot, StoryBoard.Text(section.Guest.NameKey), 42, new Vector2(0, y),
            TextAnchor.MiddleLeft, 940, 64).color = OchreBrown;
        MakeText(_listRoot, StoryBoard.Text(section.Guest.PersonaKey), 28, new Vector2(0, y - 54),
            TextAnchor.MiddleLeft, 940, 46).color = new Color(0.62f, 0.55f, 0.50f);
        y -= 130f;
        for (int r = 0; r &lt; section.Rows.Count; r++)
        {
            var row = section.Rows[r];
            var chapter = row.Chapter;
            var nameKey = section.Guest.NameKey;
            var card = MakeCard(_listRoot, "Chapter_" + chapter.Id, new Vector2(0, y),
                new Vector2(940, 96), row.IsLive
                    ? new Color(SageGreen.r, SageGreen.g, SageGreen.b, 0.16f)
                    : new Color(ColdGrey.r, ColdGrey.g, ColdGrey.b, 0.22f));
            if (row.IsLive)
            {
                var accent = MakeCard(card, "Accent", new Vector2(-452, 0), new Vector2(8, 72), SageGreen);
                var img = accent.GetComponent&lt;Image&gt;();
                img.raycastTarget = false;
                MakeText(card, StoryBoard.Text(chapter.TitleKey), 38, new Vector2(-60, 0),
                    TextAnchor.MiddleLeft, 640, 88).color = InkBrown;
                var button = card.gameObject.AddComponent&lt;Button&gt;();
                button.onClick.AddListener(() =&gt; ShowChapter(nameKey, chapter));
            }
```

```
                else
                {
                    MakeText(card, StoryBoard.Text(chapter.TitleKey), 38, new Vector2(-60, 0),
                        TextAnchor.MiddleLeft, 640, 88).color = new Color(0.56f, 0.55f, 0.53f);
                    if (_previewArmed)
                    {
                        // T014-B3: armed face - the release-week label
                        // shifts left, the preview button takes the right
                        // rail; only locked rows carry it (live rows open
                        // free, dead ends never render).
                        MakeText(card, string.Format(StoryBoard.Text(KeyLockedLabel), chapter.ReleaseWeek),
                            26, new Vector2(-160, 22), TextAnchor.MiddleLeft, 440, 44).color = ColdGrey;
                        var tapped = chapter;
                        MakeButton(card, StoryBoard.Text(KeyPreviewButton), new Vector2(330, 0),
                            new Vector2(210, 72), () =&gt; PreviewTapped?.Invoke(tapped));
                    }
                    else
                    {
                        MakeText(card, string.Format(StoryBoard.Text(KeyLockedLabel), chapter.ReleaseWeek),
                            30, new Vector2(270, 0), TextAnchor.MiddleRight, 360, 60).color = ColdGrey;
                    }
                }
                y -= 110f;
            }
            y -= 10f;
        }
        MakeButton(_listRoot, StoryBoard.Text(KeyCloseLabel), new Vector2(0, -770),
            new Vector2(420, 92), () =&gt; _overlay.gameObject.SetActive(false));
    }
    void ShowChapter(string guestNameKey, ChapterEntry chapter)
    {
        _listRoot.gameObject.SetActive(false);
        Clear(_detailRoot);
        _detailRoot.gameObject.SetActive(true);
        MakeText(_detailRoot, StoryBoard.Text(chapter.TitleKey), 62, new Vector2(0, 700),
            TextAnchor.MiddleCenter, 1020, 120).color = OchreBrown;
        MakeText(_detailRoot, "—— " + StoryBoard.Text(guestNameKey), 34, new Vector2(0, 560),
            TextAnchor.MiddleCenter, 1020, 60).color = new Color(0.55f, 0.48f, 0.42f);
        MakeText(_detailRoot, StoryBoard.Text(chapter.OpenKey), 40, new Vector2(0, 420),
            TextAnchor.MiddleCenter, 940, 220).color = InkBrown;
        float y = 150f;
        var lines = StoryBoard.ClueLines(chapter);
        for (int i = 0; i &lt; lines.Count; i++)
        {
            MakeText(_detailRoot, lines[i], 32, new Vector2(0, y),
                TextAnchor.MiddleCenter, 940, 72).color = InkBrown;
            y -= 80f;
        }
        var closeLine = MakeText(_detailRoot, StoryBoard.Text(chapter.CloseKey), 40, new Vector2(0, y - 110),
            TextAnchor.MiddleCenter, 940, 150);
```

```
        closeLine.color = new Color(0.72f, 0.58f, 0.55f);
        // T015-D: live-chapter keepsake share - the armed face splits the
        // bottom rail with the close button; disarmed keeps the centered
        // close layout (the face never renders a dead button).
        if (_storyShareArmed)
        {
            var tapped = chapter;
            MakeButton(_detailRoot, StoryBoard.Text(KeyCloseLabel), new Vector2(-250, -790),
                new Vector2(420, 84), ShowList);
            MakeButton(_detailRoot, StoryBoard.Text(KeyStoryShareButton), new Vector2(250, -790),
                new Vector2(420, 84), () =&gt; StoryShareTapped?.Invoke(tapped));
        }
        else
        {
            MakeButton(_detailRoot, StoryBoard.Text(KeyCloseLabel), new Vector2(0, -790),
                new Vector2(420, 84), ShowList);
        }
    }
    /// &lt;summary&gt;T014-B3: renders an ad-granted locked-chapter preview -
    /// title, host guest and the opening lines only. Clues and the close
    /// line stay sealed until the chapter flips live (the preview never
    /// spoils the weekly reveal), and the note names the flavor so the
    /// player always knows this is a peek, not the full chapter.&lt;/summary&gt;
    public void ShowPreview(ChapterEntry chapter)
    {
        _listRoot.gameObject.SetActive(false);
        Clear(_detailRoot);
        _detailRoot.gameObject.SetActive(true);
        MakeText(_detailRoot, StoryBoard.Text(chapter.TitleKey), 62, new Vector2(0, 700),
            TextAnchor.MiddleCenter, 1020, 120).color = OchreBrown;
        MakeText(_detailRoot, StoryBoard.Text(KeyPreviewNote), 30, new Vector2(0, 566),
            TextAnchor.MiddleCenter, 1020, 56).color = ColdGrey;
        MakeText(_detailRoot, "—— " + StoryBoard.Text(GuestNameKeyOf(chapter.GuestId)), 34,
            new Vector2(0, 486), TextAnchor.MiddleCenter, 1020, 60).color = new Color(0.55f, 0.48f, 0.42f);
        MakeText(_detailRoot, StoryBoard.Text(chapter.OpenKey), 40, new Vector2(0, 300),
            TextAnchor.MiddleCenter, 940, 240).color = InkBrown;
        MakeButton(_detailRoot, StoryBoard.Text(KeyCloseLabel), new Vector2(0, -790),
            new Vector2(420, 84), ShowList);
    }
    static string GuestNameKeyOf(int guestId)
    {
        var guests = GuestConfig.GuestsBuiltIn();
        for (int i = 0; i &lt; guests.Length; i++)
            if (guests[i].Id == guestId) return guests[i].NameKey;
        return string.Empty;
    }
    // ———— UGUI· YAML ————
    static void Clear(RectTransform root)
    {
        for (int i = root.childCount - 1; i &gt;= 0; i--)
```

```
        Destroy(root.GetChild(i).gameObject);
    }
    static RectTransform MakeContainer(RectTransform parent, string name)
    {
        var go = new GameObject(name, typeof(RectTransform));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = Vector2.zero;
        rt.offsetMin = rt.offsetMax = Vector2.zero;
        return rt;
    }
    static RectTransform MakeCard(RectTransform parent, string name, Vector2 pos, Vector2 size, Color color)
    {
        var go = new GameObject(name, typeof(RectTransform), typeof(Image));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = size;
        var img = go.GetComponent&lt;Image&gt;();
        img.sprite = MakeSquareSpriteShared();
        img.color = color;
        return rt;
    }
    static Text MakeText(RectTransform parent, string text, int size, Vector2 pos, TextAnchor anchor, float w, float
    {
        var go = new GameObject("Text", typeof(RectTransform), typeof(Text));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = new Vector2(w, h);
        var t = go.GetComponent&lt;Text&gt;();
        t.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
        t.text = text;
        t.fontSize = size;
        t.alignment = anchor;
        t.color = InkBrown;
        return t;
    }
    static void MakeButton(RectTransform parent, string label, Vector2 pos, Vector2 size, Action onClick)
    {
        var go = new GameObject("ActionButton", typeof(RectTransform), typeof(Image), typeof(Button));
        go.transform.SetParent(parent, false);
        var rt = (RectTransform)go.transform;
        rt.anchorMin = rt.anchorMax = new Vector2(0.5f, 0.5f);
        rt.anchoredPosition = pos;
        rt.sizeDelta = size;
        var img = go.GetComponent&lt;Image&gt;();
        img.sprite = MakeSquareSpriteShared();
```

```
            img.color = SageGreen;
            var txt = MakeText(rt, label, 40, Vector2.zero, TextAnchor.MiddleCenter, size.x, size.y);
            txt.color = MilkWhite;
            txt.raycastTarget = false;
            go.GetComponent&lt;Button&gt;().onClick.AddListener(() =&gt; onClick());
        }
        static Sprite _sharedSquare;
        static Sprite MakeSquareSpriteShared()
        {
            if (_sharedSquare == null)
            {
                var tex = new Texture2D(8, 8, TextureFormat.RGBA32, false);
                var px = tex.GetPixels32();
                for (int i = 0; i &lt; px.Length; i++) px[i] = new Color32(255, 255, 255, 255);
                tex.SetPixels32(px);
                tex.Apply(false, false);
                _sharedSquare = Sprite.Create(tex, new Rect(0, 0, 8, 8), new Vector2(0.5f, 0.5f), 8f);
            }
            return _sharedSquare;
        }
    }
}
// StoryPreviewOracle.cs
using System;
namespace Sy.P06.Newbie
{
    /// &lt;summary&gt;
    /// T014-B slice 3: the story_preview consumption law (ad point table row
    /// "story_preview", V1.2 cap 1/day, surface story/locked chapter). The
    /// grant flips a locked chapter into a read-only preview of its opening
    /// - the content-style rewarded point the pitch doc calls out for the
    /// female-leaning audience (higher ad tolerance on story content). The
    /// oracle owns the shared AdCapPolicy consume exactly like the speed
    /// oracle (P06's mock ad service has no cap logic); it stays pure C#
    /// with injected day keys so the whole law is headless-verifiable. The
    /// tutorial keeps the preview face hidden (zero-ad red line before M5);
    /// the formal story batch lights it via SetPreviewFace(true).
    /// &lt;/summary&gt;
    public sealed class StoryPreviewOracle
    {
        readonly AdPointEntry _point;
        readonly Sy.Platform.AdCapPolicy _adCap;
        public StoryPreviewOracle(AdPointLaw law)
            : this(law, Sy.Platform.AdCapPolicy.DefaultDailyCap) { }
        /// &lt;summary&gt;Gates construct oracles with a small global cap to pin
        /// the global/category interplay without touching the policy type
        /// from the editor assembly (CraftSpeedOracle pattern).&lt;/summary&gt;
        public StoryPreviewOracle(AdPointLaw law, int globalDailyCap)
        {
            _point = default;
```

```
            var pts = law.Points;
            for (int i = 0; i &lt; pts.Length; i++)
                if (pts[i].Id == "story_preview") { _point = pts[i]; break; }
            _adCap = new Sy.Platform.AdCapPolicy(globalDailyCap);
        }
        public string PointId =&gt; _point.Id;
        public string Category =&gt; _point.Category;
        public int DailyCap =&gt; _point.CapPerDay;
        public int UsedToday(string dayKey)
            =&gt; _adCap.UsedCategoryToday(dayKey, _point.Category);
        /// &lt;summary&gt;Pre-offer gate: only a locked chapter face with category
        /// budget left - no preview button on live rows, no dead-end taps.&lt;/summary&gt;
        public bool CanOffer(string dayKey, bool chapterLocked)
            =&gt; ValidFace(chapterLocked)
            &amp;&amp; _adCap.HasCategoryBudget(dayKey, _point.Category);
        /// &lt;summary&gt;Watched-ad grant: consumes the one story_preview slot
        /// (1/day) and reports success - the caller then opens the preview.
        /// Denies without consuming whenever the face is invalid (live
        /// chapter) or the day's budget is gone.&lt;/summary&gt;
        public bool TryGrant(string dayKey, bool chapterLocked)
        {
            if (_point.Id == null) return false;
            if (!ValidFace(chapterLocked)) return false;
            return _adCap.TryConsumeRewarded(dayKey, _point.Category);
        }
        static bool ValidFace(bool chapterLocked) =&gt; chapterLocked;
    }
}
// TipCounter.cs
using System;
using Sy.Core;
using Sy.Idle;
namespace Sy.P06.Newbie
{
    /// &lt;summary&gt;
    /// W11b offline tip counter: the shop keeps collecting tips while the
    /// player is away. Deterministic core - the clock arrives as a timestamp
    /// argument and the anchor lives in the local save slot (zero-server
    /// ruling: local save is authoritative), so the settle gate can replay
    /// any timeline. Fresh installs start the clock at first open and show
    /// zero fake tips (care/return principle: no fabricated earnings).
    /// Claiming slides the anchor forward (parallel claim, never a reset).
    /// &lt;/summary&gt;
    public sealed class TipCounter
    {
        public const string AnchorSlot = "p06_idle_tip_anchor";
        readonly ISaveService _save;
        readonly FastClaimService _claim;
        public TipCounter(IdleFormula formula, ISaveService save, long nowMs)
        {
```

```
            _save = save ?? throw new ArgumentNullException(nameof(save));
            long anchor = ParseAnchor(save.Load(AnchorSlot));
            if (anchor &lt;= 0)
            {
                anchor = nowMs;   // fresh install: the idle clock starts now
                _save.Save(AnchorSlot, anchor.ToString());
            }
            _claim = new FastClaimService(formula, anchor);
        }
        /// &lt;summary&gt;Unclaimed tips so far, without consuming them.&lt;/summary&gt;
        public double PeekPending(long nowMs) =&gt; _claim.Peek(nowMs);
        /// &lt;summary&gt;Claim tips; the anchor slides to now (the clock never resets).&lt;/summary&gt;
        public double Claim(long nowMs)
        {
            double amount = _claim.Claim(nowMs);
            _save.Save(AnchorSlot, _claim.AnchorTsMs.ToString());
            return amount;
        }
        static long ParseAnchor(string raw)
        {
            if (string.IsNullOrEmpty(raw)) return 0;
            return long.TryParse(raw, out long v) ? v : 0;
        }
    }
}
// TutorialFlow.cs
using System;
namespace Sy.P06.Newbie
{
    /// &lt;summary&gt;
    ///        evt_newbie_step 1-5 ①   ②    ③    ④    ⑤     +
    ///      ShopSim      /Mastermind               /  /    ——
    /// Play   headless   NewbieFlowCheck
    /// &lt;/summary&gt;
    public sealed class TutorialFlow
    {
        public int Step { get; private set; }
        readonly Action&lt;string&gt; _hint;
        readonly Action&lt;int&gt; _onStep;
        public TutorialFlow(ShopSim sim, Action&lt;string&gt; hint, Action&lt;int&gt; onStep)
        {
            _hint = hint ?? delegate { };
            _onStep = onStep ?? delegate { };
            sim.OrderAccepted += OnOrderAccepted;
            sim.FeedbackGiven += OnFeedbackGiven;
            sim.BlendCompleted += OnBlendCompleted;
            sim.RecipeCardShown += OnRecipeCardShown;
            sim.OrderCompleted += OnOrderCompleted;
            sim.StarGained += OnStarGained;
            sim.CodexUnlocked += OnCodexUnlocked;
```

```
    sim.Settled += OnSettled;
    sim.StateChanged += OnStateChanged;
    _hint("           ——       ");
}
void Advance(int step)
{
    if (Step &gt;= step) return;
    Step = step;
    _onStep(step);
}
void OnOrderAccepted()
{
    Advance(1);
    _hint(NewbieConfig.GuestStoryClue);   //               §4.2 ②
}
void OnFeedbackGiven(int slot, int materialId, FeedbackState state, int trueSlot)
{
    switch (state)
    {
        case FeedbackState.Correct:
            _hint(NewbieConfig.SlotNames[slot] + "    " + NewbieConfig.MaterialNames[materialId] + " ——
            break;
        case FeedbackState.Warm:
            _hint(string.Format(NewbieConfig.SnarkWarm, NewbieConfig.SlotNames[trueSlot])
                  + " " + NewbieConfig.MaterialNames[materialId] + " ");
            break;
        case FeedbackState.Cold:
            _hint(NewbieConfig.SnarkCold + " " + NewbieConfig.MaterialNames[materialId] + " ");
            break;
    }
}
void OnBlendCompleted()
{
    Advance(2);
    _hint("        " + NewbieConfig.RecipeName + "   ——    ");
}
void OnRecipeCardShown()
{
    _hint("      —— " + NewbieConfig.RecipeDesc + "                ");
}
void OnOrderCompleted(int coins, int hearts)
{
    Advance(3);
    _hint("       +" + coins + " ·   +" + hearts + "——      ");
}
void OnStarGained(int stars)
{
    _hint("   +" + NewbieConfig.OrderStars + "   " + stars + "  ——" + NewbieConfig.GuestName + "   ");
}
void OnCodexUnlocked()
```

```
        {
            Advance(4);
            _hint("         " + NewbieConfig.GuestName + "——           ");
        }
        void OnSettled()
        {
            Advance(5);
            _hint("           ——            ");
        }
        void OnStateChanged(ShopSim.ShopState state)
        {
            switch (state)
            {
                case ShopSim.ShopState.GuestArriving:
                    _hint("       ——       ");
                    break;
                case ShopSim.ShopState.WaitingAccept:
                    _hint(NewbieConfig.GuestName + "     " + NewbieConfig.RecipeName + " ——          ");
                    break;
            }
        }
    }
}
// UiInputChainTests.cs
using System.Collections;
using NUnit.Framework;
using Sy.P06.Newbie;
using UnityEngine;
using UnityEngine.EventSystems;
using UnityEngine.InputSystem;
using UnityEngine.InputSystem.UI;
using UnityEngine.UI;
using UnityEngine.TestTools;
namespace Sy.P06.PmTests
{
    /// &lt;summary&gt;
    /// X1587 (U262 batch A, 12th doc sec 12 v2.0 input-chain truth law): the
    /// P06 direct-call debt absorption slice. VirtualUserCheck's persona bot
    /// fires the accept button's onClick.Invoke() straight on an inactive
    /// GameObject - the handler path only, skipping raycast, the active /
    /// interactable gate and the EventSystem. Preparing this injection domain
    /// flushed out the real bug that seam could never see: ShowBubble(true)
    /// had ZERO call sites in the whole project, so on a real device the
    /// tutorial told the player to tap a button that never appeared and the
    /// shop soft-locked at WaitingAccept (fixed in BattleLauncher.OnStateChanged
    /// the same batch). This PlayMode domain drives the SAME accept flow with
    /// SYNTHESIZED touch input through the real UGUI pointer chain, so the
    /// shipping seat - not the delegate - is what gets proven. Rig mirrors the
    /// editor BuildRig (WorldSpace 1080x1920 canvas + ortho camera + ShopSim
    /// + ShopView) plus the P08 X1297 rig additions.
```

```
/// X1597 (slice c, X1595 design freeze section 2): the N3 random-roam
/// bot law extends this domain - 72 seeded pokes across both blind
/// domains (in-face placeholder bands + off-face screen dead zones)
/// interleaved with discovered-seat roams (accept / drawer / HUD
/// affordances), every one through the real pointer chain; zero
/// softlock, first accept reachable via the true chain, the invest
/// book moving under the roaming finger, and the X1587 seat-swallow
/// law holding for the whole roam.
/// &lt;/summary&gt;
[TestFixture]
public sealed class UiInputChainTests : InputTestFixture
{
    GameObject _root;
    Canvas _canvas;
    Camera _cam;
    ShopSim _sim;
    ShopView _view;
    Button _acceptBtn;
    int _acceptEvents;
    int _acceptDone;
    Button[] _matButtons;
    Button _storyBtn, _caravanBtn, _codexBtn, _facilityBtn, _signBtn;
    int _materialEvents;
    int _investDone;
    const int UiLayer = 5;
    public override void Setup()
    {
        base.Setup();
        _root = new GameObject("UiInputChainRig");
        var esGo = new GameObject("TestEventSystem");
        esGo.transform.SetParent(_root.transform, false);
        esGo.AddComponent&lt;EventSystem&gt;();
        esGo.AddComponent&lt;InputSystemUIInputModule&gt;();
        var camGo = new GameObject("Cam");
        camGo.transform.SetParent(_root.transform, false);
        _cam = camGo.AddComponent&lt;Camera&gt;();
        _cam.orthographic = true;
        _cam.orthographicSize = 960f;             // half the 1920 design height
        _cam.aspect = 1080f / 1920f;
        _cam.transform.position = new Vector3(0f, 0f, -100f);
        _cam.farClipPlane = 2000f;
        _cam.cullingMask = 1 &lt;&lt; UiLayer;
        var canvasGo = new GameObject("RigCanvas");
        canvasGo.transform.SetParent(_root.transform, false);
        _canvas = canvasGo.AddComponent&lt;Canvas&gt;();
        _canvas.renderMode = RenderMode.WorldSpace;
        _canvas.worldCamera = _cam;
        ((RectTransform)canvasGo.transform).sizeDelta = new Vector2(1080f, 1920f);
        canvasGo.AddComponent&lt;GraphicRaycaster&gt;();
        _sim = new ShopSim();
```

```
var viewGo = new GameObject("ShopView");
viewGo.transform.SetParent(_root.transform, false);
_view = viewGo.AddComponent&lt;ShopView&gt;();
_view.Initialize(_canvas, _sim);
SetLayer(_root, UiLayer);
// Host glue in the launcher shape (BattleLauncher parity):
// view event -&gt; handler -&gt; sim op.
_view.AcceptRequested += () =&gt;
{
    _acceptEvents++;
    if (_sim.AcceptOrder()) _acceptDone++;
};
// Launcher state faces the accept flow rides on - the exact
// OnStateChanged wiring BattleLauncher now ships (WaitingAccept
// shows the bubble; Crafting swaps panels and hides it).
_sim.StateChanged += state =&gt;
{
    switch (state)
    {
        case ShopSim.ShopState.GuestArriving:
            _view.HideOpeningOverlay();
            break;
        case ShopSim.ShopState.WaitingAccept:
            _view.ShowBubble(true);
            break;
        case ShopSim.ShopState.Crafting:
            _view.ShowCraftingPanel(true);
            _view.ShowBubble(false);
            break;
    }
};
_acceptBtn = FindButton("\u63a5\u5355");
// Ledger hygiene (P02 N3 law): NUnit can reuse one fixture
// instance across the tests in this fixture, so every counter
// the roam asserts on is zeroed per test.
_acceptEvents = 0;
_acceptDone = 0;
_materialEvents = 0;
_investDone = 0;
// Launcher parity (BattleLauncher.OnMaterialTapped): a drawer
// seat's tap routes into the sim invest op through the view
// event - the seam the N3 roam proves with a real finger.
_view.MaterialTapped += id =&gt;
{
    _materialEvents++;
    if (_sim.TryInvest(id)) _investDone++;
};
// Discovered seat inventory for the N3 roam: the accept seat,
// the material drawer and the HUD affordances the launcher
// ships. Live or dead, the pointer chain's active/interactable
```

```
        // gate decides what the finger does - never the bot.
        _matButtons = new Button[NewbieConfig.MaterialCount];
        for (int i = 0; i &lt; NewbieConfig.MaterialCount; i++)
            _matButtons[i] = FindButton("Mat_" + i);
        _storyBtn = FindButton("StoryButton");
        _caravanBtn = FindButton("CaravanButton");
        _codexBtn = FindButton("CodexButton");
        _facilityBtn = FindButton("FacilityButton");
        _signBtn = FindButton("SignInButton");
    }
    public override void TearDown()
    {
        if (_root != null) Object.DestroyImmediate(_root);
        _root = null;
        _canvas = null;
        _cam = null;
        _sim = null;
        _view = null;
        _acceptBtn = null;
        _matButtons = null;
        _storyBtn = null;
        _caravanBtn = null;
        _codexBtn = null;
        _facilityBtn = null;
        _signBtn = null;
        base.TearDown();
    }
    // ——— the accept seam (VirtualUserCheck persona Tap) through the
    // real pointer chain. Two faces: (1) the live seat at WaitingAccept
    // must SHIP and a synthesized tap at its runtime screen point must
    // land the accept exactly like a finger; (2) once the order is taken
    // the bubble hides and the SAME tap at the SAME screen point must be
    // swallowed by the pointer chain - the exact active/visibility gate
    // the old onClick.Invoke() seam bypassed (it fired the delegate on an
    // inactive GameObject without the engine ever seeing a pointer).
    // ———
    [UnityTest]
    public IEnumerator AcceptFlow_InjectedTouch_WalksRealPointerChain()
    {
        Assert.NotNull(_acceptBtn, "the view must build the accept seat");
        // Walk the shop to WaitingAccept the way the editor bot does
        // (sim ticks; the StateChanged glue shows the bubble on arrival).
        int guard = 0;
        while (_sim.State != ShopSim.ShopState.WaitingAccept &amp;&amp; guard++ &lt; 600)
            _sim.Tick(0.1f);
        Assert.AreEqual(ShopSim.ShopState.WaitingAccept, _sim.State,
            "the tutorial shop reaches the accept moment");
        Assert.IsTrue(_acceptBtn.gameObject.activeInHierarchy,
            "the accept seat SHIPS at WaitingAccept - the X1587 soft-lock face");
        Assert.IsTrue(_acceptBtn.interactable, "the live accept seat is tappable");
```

```
    yield return TouchTap(_acceptBtn);
    Assert.AreEqual(1, _acceptEvents, "the injected touch reached the view event");
    Assert.AreEqual(1, _acceptDone, "the handler routed the accept into the sim op");
    Assert.AreEqual(ShopSim.ShopState.Crafting, _sim.State,
        "the tap walked the whole chain: event -&gt; handler -&gt; sim op (order taken)");
    Assert.IsFalse(_acceptBtn.gameObject.activeInHierarchy,
        "the taken order hides the bubble (launcher Crafting face)");
    // The gate face the old seam could never prove: the same finger
    // tap at the same screen point now hits nothing - the pointer
    // chain swallows it before the onClick delegate is ever reached.
    yield return TouchTap(_acceptBtn);
    Assert.AreEqual(1, _acceptEvents, "the hidden seat swallowed the injected tap");
    Assert.AreEqual(1, _acceptDone, "no second accept happened through the hidden seat");
}
// ──── N3 (X1595 design freeze section 2, slice c): the random-roam
// bot law. A seeded roam - not a script: blind points across the
// whole face (placeholder bands, dead corners, off-face screen)
// interleaved with pokes at the discovered seats, every one through
// the real pointer chain. Faces: zero softlock (every poke lands
// inside the frame guard and no unexpected log survives); first
// accept reachable via the true chain (the opening beat taps the
// live accept seat and the order lands); the invest book moves
// under the roaming finger (the drawer seat routes a real invest
// during Crafting); the X1587 seat-swallow law holds for the whole
// roam (the taken order's hidden accept seat never fires a second
// accept no matter how often the roaming finger returns to it).
// ────
[UnityTest]
public IEnumerator N3_RandomRoamBot_ZeroSoftlock_FirstAcceptReachable_InvestBookMoves()
{
    const int TotalPokes = 72;   // freeze law: &gt;= 60 pokes on the rig
    const float Dt = 0.05f;      // fixed sim step, editor-check parity
    var rng = new System.Random(20261001);
    // Discovered-seat inventory must ship before any roaming: the
    // accept seat, the drawer and the HUD affordances.
    for (int i = 0; i &lt; _matButtons.Length; i++)
        Assert.NotNull(_matButtons[i], "the drawer seat Mat_" + i + " ships");
    Assert.NotNull(_storyBtn, "the story seat ships");
    Assert.NotNull(_caravanBtn, "the caravan seat ships");
    Assert.NotNull(_codexBtn, "the codex seat ships");
    Assert.NotNull(_facilityBtn, "the facility seat ships");
    Assert.NotNull(_signBtn, "the sign seat ships");
    var seats = new[]
    {
        _acceptBtn, _matButtons[0], _matButtons[1], _matButtons[2], _matButtons[3],
        _storyBtn, _caravanBtn, _codexBtn, _facilityBtn, _signBtn
    };
    // Opening beat precondition: the true state chain carries the
    // rig to the accept moment (Opening -&gt; GuestArriving -&gt;
    // WaitingAccept), exactly the walk the launcher's Update runs.
```

```
int walkGuard = 0;
while (_sim.State != ShopSim.ShopState.WaitingAccept &amp;amp;&amp;amp; walkGuard++ &lt; 600)
    _sim.Tick(Dt);
Assert.AreEqual(ShopSim.ShopState.WaitingAccept, _sim.State,
    "the true chain carried the rig to WaitingAccept");
_view.SyncView(_sim);
Assert.IsTrue(_acceptBtn.gameObject.activeInHierarchy,
    "the accept seat SHIPS at WaitingAccept - the X1587 soft-lock face");
int blindPokes = 0, inFaceBlind = 0, offFaceBlind = 0;
int landed = 0;   // loop counter - proves the roam never wedged
// The 1080x1920 face rect in screen space, so in-face blind pokes
// can land on placeholder bands (HUD strips, slot rows, counter)
// too, and off-face pokes explore the screen dead zones outside
// the world-space canvas.
var faceMin = RectTransformUtility.WorldToScreenPoint(_cam, new Vector3(-540f, -960f, 0f));
var faceMax = RectTransformUtility.WorldToScreenPoint(_cam, new Vector3(540f, 960f, 0f));
for (int poke = 0; poke &lt; TotalPokes; poke++)
{
    Vector2 point;
    if (poke == 0)
    {
        // Opening beat: the live first-accept seat is inside the
        // roam's poke domain by construction.
        point = SeatPoint(_acceptBtn);
    }
    else if (poke == 3)
    {
        // Drawer beat: a discovered material seat poked while
        // Crafting is live must route a real invest through the
        // pointer chain. Musk (id 3) is the out-of-recipe cold
        // demo - the guided strategy's first invest (L3 parity),
        // so the outcome is deterministic, never RNG-handed.
        point = SeatPoint(_matButtons[3]);
    }
    else if (poke % 3 == 0)
    {
        // Discovered-seat roam - live or dead, the pointer
        // chain's active/interactable gate decides what the
        // finger does, not the bot.
        point = SeatPoint(seats[rng.Next(seats.Length)]);
    }
    else
    {
        blindPokes++;
        bool inFace = faceMax.x &gt; faceMin.x &amp;&amp; faceMax.y &gt; faceMin.y &amp;&amp; (rng.Next() &a
        if (inFace)
        {
            inFaceBlind++;
            point = new Vector2(faceMin.x + (float)rng.NextDouble() * (faceMax.x - faceMin.x),
                                faceMin.y + (float)rng.NextDouble() * (faceMax.y - faceMin.y));
```

```
            }
            else
            {
                offFaceBlind++;
                point = new Vector2(rng.Next(Mathf.Max(1, Screen.width)), rng.Next(Mathf.Max(1, Screen.height)))
            }
        }
        int acceptBefore = _acceptEvents;
        int materialBefore = _materialEvents;
        yield return TouchTapAt(point);
        if (poke == 0)
        {
            Assert.AreEqual(1, _acceptEvents - acceptBefore,
                "opening beat: the live seat routed the finger to the view event");
            Assert.AreEqual(1, _acceptDone, "the first accept landed through the real pointer chain");
            Assert.AreEqual(ShopSim.ShopState.Crafting, _sim.State,
                "first accept reachable: the order is taken at the opening beat");
        }
        if (poke == 3)
        {
            Assert.GreaterOrEqual(_materialEvents - materialBefore, 1,
                "drawer beat: the discovered material seat routed the finger to the view event");
            Assert.GreaterOrEqual(_investDone, 1,
                "the invest book moved under the roaming finger (real pointer chain)");
            Assert.GreaterOrEqual(_sim.ColdInvests + _sim.WarmInvests + _sim.CorrectInvests, 1,
                "the routed tap landed a real invest in the sim");
        }
        // The shop clock runs under the roaming finger (fixed step
        // keeps the roam deterministic across machines).
        for (int f = 0; f &lt; 8; f++)
        {
            _sim.Tick(Dt);
            yield return null;
        }
        _view.SyncView(_sim);
        landed++;
    }
    Assert.AreEqual(TotalPokes, landed,
        "zero softlock: the roam completed every poke inside the frame guard");
    Assert.GreaterOrEqual(blindPokes, 30,
        "blind pokes carried at least a third of the roam (got " + blindPokes + ")");
    Assert.Greater(inFaceBlind, 0,
        "both blind domains exercised: in-face placeholder bands (got " + inFaceBlind + ")");
    Assert.Greater(offFaceBlind, 0,
        "both blind domains exercised: off-face screen dead zones (got " + offFaceBlind + ")");
    Assert.AreEqual(1, _acceptEvents,
        "the X1587 seat-swallow law held the whole roam: the taken order's hidden seat never fired a second acce
    Assert.AreNotEqual(ShopSim.ShopState.WaitingAccept, _sim.State,
        "the shop moved past the accept moment under the roaming finger");
    LogAssert.NoUnexpectedReceived();
```

```
        }
        // ——— rig helpers ———
        Button FindButton(string name)
        {
            foreach (var b in _canvas.GetComponentsInChildren&lt;Button&gt;(true))
                if (b.gameObject.name == name) return b;
            return null;
        }
        /// &lt;summary&gt;Runtime screen point of a seat's center (world-space rig).&lt;/summary&gt;
        Vector2 SeatPoint(Button target)
        {
            var rt = (RectTransform)target.transform;
            return RectTransformUtility.WorldToScreenPoint(_cam, rt.position);
        }
        /// &lt;summary&gt;One synthesized finger tap at an arbitrary screen point -
        /// the roam variant of TouchTap (down, one frame, up, two settle frames).&lt;/summary&gt;
        IEnumerator TouchTapAt(Vector2 screenPos)
        {
            Assert.Greater(Screen.width, 0, "player has no screen face in this run");
            Assert.Greater(Screen.height, 0, "player has no screen face in this run");
            BeginTouch(1, screenPos);
            yield return null;
            EndTouch(1, screenPos);
            yield return null;
            yield return null;
        }
        static void SetLayer(GameObject root, int layer)
        {
            root.layer = layer;
            foreach (Transform tr in root.GetComponentsInChildren&lt;Transform&gt;(true))
                tr.gameObject.layer = layer;
        }
        /// &lt;summary&gt;One synthesized finger tap at the target's runtime screen
        /// point: touch down, one frame, touch up, then two frames so the
        /// pointer chain (module update -&gt; EventSystem -&gt; onClick) settles
        /// before the caller asserts.&lt;/summary&gt;
        IEnumerator TouchTap(Button target)
        {
            var rt = (RectTransform)target.transform;
            var screenPos = RectTransformUtility.WorldToScreenPoint(_cam, rt.position);
            Assert.Greater(Screen.width, 0, "player has no screen face in this run");
            Assert.Greater(Screen.height, 0, "player has no screen face in this run");
            BeginTouch(1, screenPos);
            yield return null;
            EndTouch(1, screenPos);
            yield return null;
            yield return null;
        }
    }
}
```